

09 회귀와 예측 & 분류





강의 내용

Contents of Lecture

기간	내용	과제
01주차	오리엔테이션 lpython - 파이썬에 날개를 달자	실습 과제 (채점 X)
02주차	Numpy 소개	실습 과제
03주차	Pandas로 데이터 가공하기	실습 과제
04주차	Matplotlib를 활용한 시각화	실습 과제
05주차	탐색적 데이터 분석	실습 과제
06주차	데이터의 표본분포	실습 과제
07주차	통계적 실험과 유의성 검정	실습 과제
08주차	중간고사 - 오프라인 오픈북으로 진행	기말 프로젝트 상세 공지

기간	내용	과제
09주차	인공지능 & 회귀/예측	실습 과제
10주차	회귀와 예측 / 분류	실습 과제
11주차	분류 및 머신러닝 기초	실습 과제
12주차	통계적 머신러닝	
13주차	비지도 학습	실습 과제
14주차	알고보면 쓸모있는 신기한 기계학습	프로젝트 레포트 제출
15주차	[기말고사] Team별 프로젝트 결과 발표	종강~!

1부: 회귀와 예측

- 1) 회귀방정식 해석하기
- 2) 회귀분석 진단하기
- 3) 회귀분석 (다항 / 스플라인)





Interpreting the Regression Equation

In data science, the most important use of regression is to predict some dependent (outcome) variable. In some cases, however, gaining insight from the equation itself to understand the nature of the relationship between the predictors and the outcome can be of value. This section provides guidance on examining the regression equation and interpreting it.

Key Terms for Interpreting the Regression Equation

Correlated variables

When the predictor variables are highly correlated, it is difficult to interpret the individual coefficients.

Multicollinearity

When the predictor variables have perfect, or near-perfect, correlation, the regression can be unstable or impossible to compute.

Synonym

collinearity

Confounding variables

An important predictor that, when omitted, leads to spurious relationships in a regression equation.

Main effects

The relationship between a predictor and the outcome variable, independent of other variables.

Interactions

An interdependent relationship between two or more predictors and the response.



회귀방정식 해석하기 > Correlated Predictors

Interpreting the Regression Equation

Correlated Predictors

In multiple regression, the predictor variables are often correlated with each other. As an example, examine the regression coefficients for the model `step_lm`, fit in “Model Selection and Stepwise Regression” on page 156.

R:

```
step_lm$coefficients
```

(Intercept)	SqFtTotLiving	Bathrooms
6.178645e+06	1.992776e+02	4.239616e+04
Bedrooms	BldgGrade	PropertyTypeSingle Family
-5.194738e+04	1.371596e+05	2.291206e+04
PropertyTypeTownhouse	SqFtFinBasement	YrBuilt
8.447916e+04	7.046975e+00	-3.565425e+03

Python:

```
print(f'Intercept: {best_model.intercept_:.3f}')
print('Coefficients:')
for name, coef in zip(best_variables, best_model.coef_):
    print(f' {name}: {coef}')
```

The coefficient for `Bedrooms` is negative! This implies that adding a bedroom to a house will reduce its value. How can this be? This is because the predictor variables are correlated: larger houses tend to have more bedrooms, and it is the size that drives house value, not the number of bedrooms. Consider two homes of the exact same size: it is reasonable to expect that a home with more but smaller bedrooms would be considered less desirable.

Having correlated predictors can make it difficult to interpret the sign and value of regression coefficients (and can inflate the standard error of the estimates). The variables for bedrooms, house size, and number of bathrooms are all correlated. This is illustrated by the following example in R, which fits another regression removing the variables `SqFtTotLiving`, `SqFtFinBasement`, and `Bathrooms` from the equation:

```
update(step_lm, . ~ . - SqFtTotLiving - SqFtFinBasement - Bathrooms)
```

Call:

```
lm(formula = AdjSalePrice ~ Bedrooms + BldgGrade + PropertyType +
    YrBuilt, data = house, na.action = na.omit)
```

Coefficients:

(Intercept)	Bedrooms
4913973	27151
BldgGrade	PropertyTypeSingle Family
248998	-19898
PropertyTypeTownhouse	YrBuilt
-47355	-3212



회귀방정식 해석하기 > *Correlated Predictors*

Interpreting the Regression Equation

The `update` function can be used to add or remove variables from a model. Now the coefficient for bedrooms is positive—in line with what we would expect (though it is really acting as a proxy for house size, now that those variables have been removed).

In *Python*, there is no equivalent to *R*'s `update` function. We need to refit the model with the modified predictor list:

```
predictors = ['Bedrooms', 'BldgGrade', 'PropertyType', 'YrBuilt']
outcome = 'AdjSalePrice'

X = pd.get_dummies(house[predictors], drop_first=True)

reduced_lm = LinearRegression()
reduced_lm.fit(X, house[outcome])
```

Correlated variables are only one issue with interpreting regression coefficients. In `house_lm`, there is no variable to account for the location of the home, and the model is mixing together very different types of regions. Location may be a *confounding* variable; see “**Confounding Variables**” on page 172 for further discussion.



회귀방정식 해석하기 > Multicollinearity

Interpreting the Regression Equation

Multicollinearity

An extreme case of correlated variables produces multicollinearity—a condition in which there is redundancy among the predictor variables. Perfect multicollinearity occurs when one predictor variable can be expressed as a linear combination of others. Multicollinearity occurs when:

- A variable is included multiple times by error.
- P dummies, instead of $P - 1$ dummies, are created from a factor variable (see “Factor Variables in Regression” on page 163).
- Two variables are nearly perfectly correlated with one another.

Multicollinearity in regression must be addressed—variables should be removed until the multicollinearity is gone. A regression does not have a well-defined solution in the presence of perfect multicollinearity. Many software packages, including *R* and *Python*, automatically handle certain types of multicollinearity. For example, if `SqFtTotLiving` is included twice in the regression of the house data, the results are the same as for the `house_lm` model. In the case of nonperfect multicollinearity, the software may obtain a solution, but the results may be unstable.



Multicollinearity is not such a problem for nonlinear regression methods like trees, clustering, and nearest-neighbors, and in such methods it may be advisable to retain P dummies (instead of $P - 1$). That said, even in those methods, nonredundancy in predictor variables is still a virtue.

Confounding Variables

With correlated variables, the problem is one of commission: including different variables that have a similar predictive relationship with the response. With *confounding variables*, the problem is one of omission: an important variable is not included in the regression equation. Naive interpretation of the equation coefficients can lead to invalid conclusions.

Take, for example, the King County regression equation `house_lm` from “[Example: King County Housing Data](#)” on page 151. The regression coefficients of `SqFtLot`, `Bathrooms`, and `Bedrooms` are all negative. The original regression model does not contain a variable to represent location—a very important predictor of house price.

To model location, include a variable `ZipGroup` that categorizes the zip code into one of five groups, from least expensive (1) to most expensive (5):⁵

```
lm(formula = AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +
    Bedrooms + BldgGrade + PropertyType + ZipGroup, data = house,
    na.action = na.omit)
```

```
Coefficients:
      (Intercept)      SqFtTotLiving
      -6.666e+05      2.186e+02
      SqFtLot
      4.550e-01      5.928e+03
      Bedrooms
      -4.168e+04      9.854e+04
PropertyTypeSingle Family  PropertyTypeTownhouse
      1.932e+04      -7.820e+04
      ZipGroup2
      5.332e+04      1.163e+05
      ZipGroup4
      1.784e+05      3.384e+05
      ZipGroup3
```

The same model in *Python*:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms',
              'BldgGrade', 'PropertyType', 'ZipGroup']
outcome = 'AdjSalePrice'

X = pd.get_dummies(house[predictors], drop_first=True)

confounding_lm = LinearRegression()
confounding_lm.fit(X, house[outcome])

print(f'Intercept: {confounding_lm.intercept_:.3f}')
print('Coefficients:')
for name, coef in zip(X.columns, confounding_lm.coef_):
    print(f' {name}: {coef}')
```




회귀방정식 해석하기 > *Confounding Variables*

Interpreting the Regression Equation

ZipGroup is clearly an important variable: a home in the most expensive zip code group is estimated to have a higher sales price by almost \$340,000. The coefficients of SqFtLot and Bathrooms are now positive, and adding a bathroom increases the sale price by \$5,928.

The coefficient for Bedrooms is still negative. While this is unintuitive, this is a well-known phenomenon in real estate. For homes of the same livable area and number of bathrooms, having more and therefore smaller bedrooms is associated with less valuable homes.

5 There are 80 zip codes in King County, several with just a handful of sales. An alternative to directly using zip code as a factor variable, ZipGroup clusters similar zip codes into a single group. See “**Factor Variables with Many Levels**” on page 167 for details.



회귀방정식 해석하기 > Interactions and Main Effects

Interpreting the Regression Equation

Interactions and Main Effects

Statisticians like to distinguish between *main effects*, or independent variables, and the *interactions* between the main effects. Main effects are what are often referred to as the *predictor variables* in the regression equation. An implicit assumption when only main effects are used in a model is that the relationship between a predictor variable and the response is independent of the other predictor variables. This is often not the case.

For example, the model fit to the King County Housing Data in “[Confounding Variables](#)” on [page 172](#) includes several variables as main effects, including ZipCode. Location in real estate is everything, and it is natural to presume that the relationship between, say, house size and the sale price depends on location. A big house built in a low-rent district is not going to retain the same value as a big house built in an expensive area. You include interactions between variables in R using the * operator. For the King County data, the following fits an interaction between SqFtTotLiving and ZipGroup:

```
lm(formula = AdjSalePrice ~ SqFtTotLiving * ZipGroup + SqFtLot +  
    Bathrooms + Bedrooms + BldgGrade + PropertyType, data = house,  
    na.action = na.omit)
```

Coefficients:

(Intercept)	SqFtTotLiving
-4.853e+05	1.148e+02
ZipGroup2	ZipGroup3
-1.113e+04	2.032e+04
ZipGroup4	ZipGroup5
2.050e+04	-1.499e+05
SqFtLot	Bathrooms
6.869e-01	-3.619e+03
Bedrooms	BldgGrade
-4.180e+04	1.047e+05
PropertyTypeSingle Family	PropertyTypeTownhouse
1.357e+04	-5.884e+04
SqFtTotLiving:ZipGroup2	SqFtTotLiving:ZipGroup3
3.260e+01	4.178e+01
SqFtTotLiving:ZipGroup4	SqFtTotLiving:ZipGroup5
6.934e+01	2.267e+02

The resulting model has four new terms: SqFtTotLiving:ZipGroup2, SqFtTotLiving:ZipGroup3, and so on.



회귀방정식 해석하기 > Interactions and Main Effects

Interpreting the Regression Equation

In *Python*, we need to use the `statsmodels` package to train linear regression models with interactions. This package was designed similar to *R* and allows defining models using a formula interface:

```
model = smf.ols(formula='AdjSalePrice ~ SqFtTotLiving*ZipGroup + SqFtLot + ' +  
                'Bathrooms + Bedrooms + BldgGrade + PropertyType', data=house)  
results = model.fit()  
results.summary()
```

The `statsmodels` package takes care of categorical variables (e.g., `ZipGroup[T.1]`, `PropertyType[T.Single Family]`) and interaction terms (e.g., `SqFtTotLiving:ZipGroup[T.1]`).

Location and house size appear to have a strong interaction. For a home in the lowest `ZipGroup`, the slope is the same as the slope for the main effect `SqFtTotLiving`, which is \$118 per square foot (this is because *R* uses *reference* coding for factor variables; see “[Factor Variables in Regression](#)” on page 163). For a home in the highest `ZipGroup`, the slope is the sum of the main effect plus `SqFtTotLiving:ZipGroup5`, or \$115 + \$227 = \$342 per square foot. In other words, adding a square foot in the most expensive zip code group boosts the predicted sale price by a factor of almost three, compared to the average boost from adding a square foot.



Model Selection with Interaction Terms

In problems involving many variables, it can be challenging to decide which interaction terms should be included in the model. Several different approaches are commonly taken:

- In some problems, prior knowledge and intuition can guide the choice of which interaction terms to include in the model.
- Stepwise selection (see “[Model Selection and Stepwise Regression](#)” on page 156) can be used to sift through the various models.
- Penalized regression can automatically fit to a large set of possible interaction terms.
- Perhaps the most common approach is to use *tree models*, as well as their descendants, *random forest* and *gradient boosted trees*. This class of models automatically searches for optimal interaction terms; see “[Tree Models](#)” on page 249.

Key Ideas

- Because of correlation between predictors, care must be taken in the interpretation of the coefficients in multiple linear regression.
- Multicollinearity can cause numerical instability in fitting the regression equation.
- A confounding variable is an important predictor that is omitted from a model and can lead to a regression equation with spurious relationships.
- An interaction term between two variables is needed if the relationship between the variables and the response is interdependent.



Regression Diagnostics

In explanatory modeling (i.e., in a research context), various steps, in addition to the metrics mentioned previously (see [“Assessing the Model” on page 153](#)), are taken to assess how well the model fits the data; most are based on analysis of the residuals. These steps do not directly address predictive accuracy, but they can provide useful insight in a predictive setting.

Key Terms for Regression Diagnostics

Standardized residuals

Residuals divided by the standard error of the residuals.

Outliers

Records (or outcome values) that are distant from the rest of the data (or the predicted outcome).

Influential value

A value or record whose presence or absence makes a big difference in the regression equation.

Leverage

The degree of influence that a single record has on a regression equation.

Synonym

hat-value

Non-normal residuals

Non-normally distributed residuals can invalidate some technical requirements of regression but are usually not a concern in data science.

Heteroskedasticity

When some ranges of the outcome experience residuals with higher variance (may indicate a predictor missing from the equation).

Partial residual plots

A diagnostic plot to illuminate the relationship between the outcome variable and a single predictor.

Synonym

added variables plot



Outliers

Generally speaking, an extreme value, also called an *outlier*, is one that is distant from most of the other observations. Just as outliers need to be handled for estimates of location and variability (see “[Estimates of Location](#)” on page 7 and “[Estimates of Variability](#)” on page 13), outliers can cause problems with regression models. In regression, an outlier is a record whose actual y value is distant from the predicted value. You can detect outliers by examining the *standardized residual*, which is the residual divided by the standard error of the residuals.

There is no statistical theory that separates outliers from nonoutliers. Rather, there are (arbitrary) rules of thumb for how distant from the bulk of the data an observation needs to be in order to be called an outlier. For example, with the boxplot, outliers are those data points that are too far above or below the box boundaries (see “[Percentiles and Boxplots](#)” on page 20), where “too far” = “more than 1.5 times the interquartile range.” In regression, the standardized residual is the metric that is typically used to determine whether a record is classified as an outlier. Standardized residuals can be interpreted as “the number of standard errors away from the regression line.”

Let’s fit a regression to the King County house sales data for all sales in zip code 98105 in R:

```
house_98105 <- house[house$ZipCode == 98105,]  
lm_98105 <- lm(AdjSalePrice ~ SqFtTotLiving + SqFtLot + Bathrooms +  
              Bedrooms + BldgGrade, data=house_98105)
```

In Python:

```
house_98105 = house.loc[house['ZipCode'] == 98105, ]  
  
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms', 'BldgGrade']  
outcome = 'AdjSalePrice'  
  
house_outlier = sm.OLS(house_98105[outcome],  
                       house_98105[predictors].assign(const=1))  
result_98105 = house_outlier.fit()
```

We extract the standardized residuals in R using the `rstandard` function and obtain the index of the smallest residual using the `order` function:

```
sresid <- rstandard(lm_98105)  
idx <- order(sresid)  
sresid[idx[1]]  
      28429  
     -4.326732
```



회귀분석 진단하기 > Outliers

Regression Diagnostics

In statsmodels, use OLSInfluence to analyze the residuals:

```
influence = OLSInfluence(result_98105)
sresiduals = influence.resid_studentized_internal
sresiduals.idxmin(), sresiduals.min()
```

The biggest overestimate from the model is more than four standard errors above the regression line, corresponding to an overestimate of \$757,754. The original data record corresponding to this outlier is as follows in R:

```
house_98105[idx[1], c('AdjSalePrice', 'SqFtTotLiving', 'SqFtLot',
                      'Bathrooms', 'Bedrooms', 'BldgGrade')]
```

AdjSalePrice	SqFtTotLiving	SqFtLot	Bathrooms	Bedrooms	BldgGrade
(dbl)	(int)	(int)	(dbl)	(int)	(int)
20429	119748	2900	7276	3	6

In Python:

```
outlier = house_98105.loc[sresiduals.idxmin(), :]
print('AdjSalePrice', outlier[outcome])
print(outlier[predictors])
```

In this case, it appears that there is something wrong with the record: a house of that size typically sells for much more than \$119,748 in that zip code. **Figure 4-4** shows an excerpt from the statutory deed from this sale: it is clear that the sale involved only partial interest in the property. In this case, the outlier corresponds to a sale that is anomalous and should not be included in the regression. Outliers could also be the result of other problems, such as a “fat-finger” data entry or a mismatch of units (e.g., reporting a sale in thousands of dollars rather than simply in dollars).

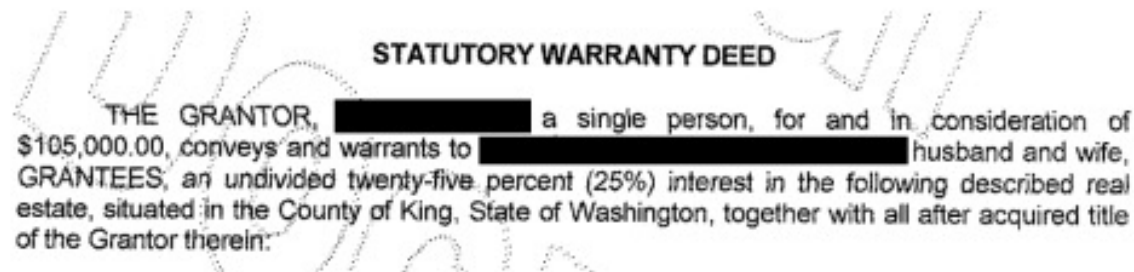


Figure 4-4. Statutory warranty deed for the largest negative residual

For big data problems, outliers are generally not a problem in fitting the regression to be used in predicting new data. However, outliers are central to anomaly detection, where finding outliers is the whole point. The outlier could also correspond to a case of fraud or an accidental action. In any case, detecting outliers can be a critical business need.



Influential Values

A value whose absence would significantly change the regression equation is termed an *influential observation*. In regression, such a value need not be associated with a large residual. As an example, consider the regression lines in Figure 4-5. The solid line corresponds to the regression with all the data, while the dashed line corresponds to the regression with the point in the upper-right corner removed. Clearly, that data value has a huge influence on the regression even though it is not associated with a large outlier (from the full regression). This data value is considered to have high *leverage* on the regression.

In addition to standardized residuals (see “Outliers” on page 177), statisticians have developed several metrics to determine the influence of a single record on a regression. A common measure of leverage is the *hat-value*; values above $2(P + 1)/n$ indicate a high-leverage data value.⁶

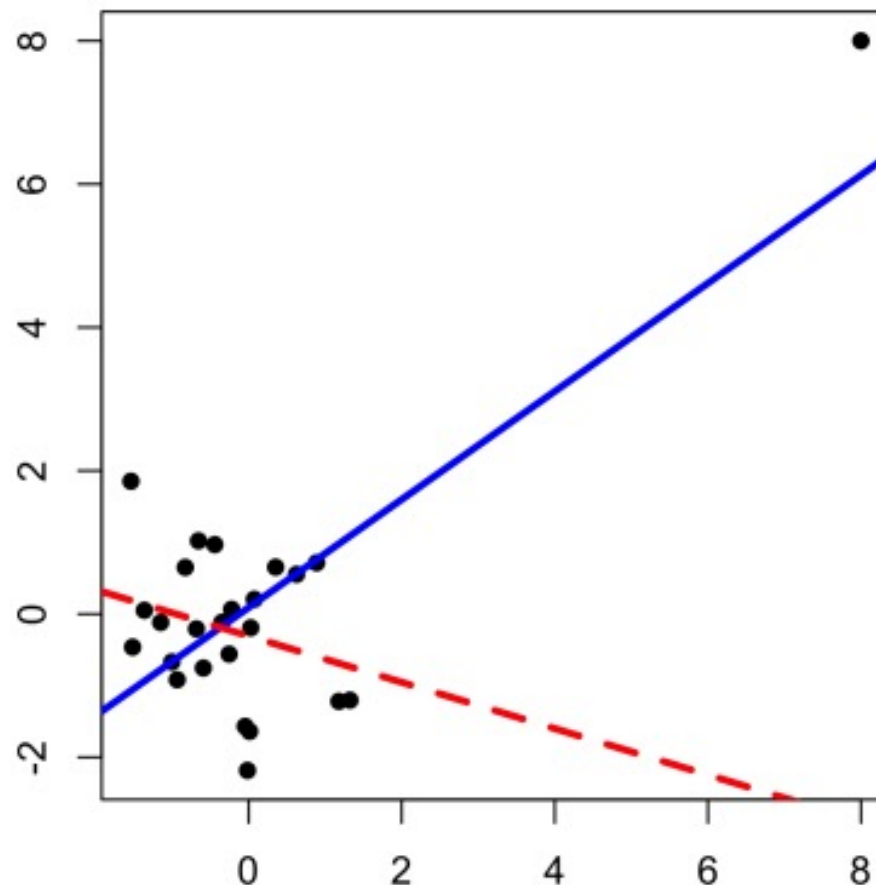


Figure 4-5. An example of an influential data point in regression

⁶ The term *hat-value* comes from the notion of the hat matrix in regression. Multiple linear regression can be expressed by the formula $\hat{Y} = HY$ where H is the hat matrix. The hat-values correspond to the diagonal of H .



회귀분석 진단하기 > Influential Values

Regression Diagnostics

Another metric is *Cook's distance*, which defines influence as a combination of leverage and residual size. A rule of thumb is that an observation has high influence if Cook's distance exceeds $4/(n - P - 1)$.

An *influence plot* or *bubble plot* combines standardized residuals, the hat-value, and Cook's distance in a single plot. **Figure 4-6** shows the influence plot for the King County house data and can be created by the following R code:

```
std_resid <- rstandard(lm_98105)
cooks_D <- cooks.distance(lm_98105)
hat_values <- hatvalues(lm_98105)
plot(subset(hat_values, cooks_D > 0.08), subset(std_resid, cooks_D > 0.08),
     xlab='hat_values', ylab='std_resid',
     cex=10*sqrt(subset(cooks_D, cooks_D > 0.08)), pch=16, col='lightgrey')
points(hat_values, std_resid, cex=10*sqrt(cooks_D))
abline(h=c(-2.5, 2.5), lty=2)
```

Here is the *Python* code to create a similar figure:

```
influence = OLSInfluence(result_98105)
fig, ax = plt.subplots(figsize=(5, 5))
ax.axhline(-2.5, linestyle='--', color='C1')
ax.axhline(2.5, linestyle='--', color='C1')
ax.scatter(influence.hat_matrix_diag, influence.resid_studentized_internal,
           s=1000 * np.sqrt(influence.cooks_distance[0]),
           alpha=0.5)
ax.set_xlabel('hat values')
ax.set_ylabel('studentized residuals')
```

There are apparently several data points that exhibit large influence in the regression. Cook's distance can be computed using the function `cooks.distance`, and you can use `hatvalues` to compute the diagnostics. The hat values are plotted on the x-axis, the residuals are plotted on the y-axis, and the size of the points is related to the value of Cook's distance.



회귀분석 진단하기 > Influential Values

Regression Diagnostics

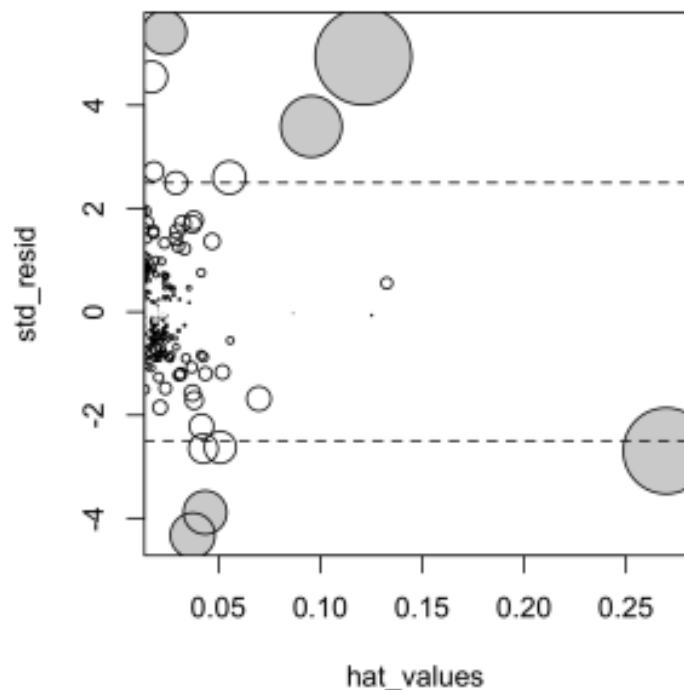


Figure 4-6. A plot to determine which observations have high influence; points with Cook's distance greater than 0.08 are highlighted in grey

Table 4-2 compares the regression with the full data set and with highly influential data points removed (Cook's distance > 0.08).

The regression coefficient for Bathrooms changes quite dramatically.⁷

Table 4-2. Comparison of regression coefficients with the full data and with influential data removed

	Original	Influential removed
(Intercept)	-772,550	-647,137
SqFtTotLiving	210	230
SqFtLot	39	33
Bathrooms	2282	-16,132
Bedrooms	-26,320	-22,888
BldgGrade	130,000	114,871

For purposes of fitting a regression that reliably predicts future data, identifying influential observations is useful only in smaller data sets. For regressions involving many records, it is unlikely that any one observation will carry sufficient weight to cause extreme influence on the fitted equation (although the regression may still have big outliers). For purposes of anomaly detection, though, identifying influential observations can be very useful.

⁷ The coefficient for Bathrooms becomes negative, which is unintuitive. Location has not been taken into account, and the zip code 98105 contains areas of disparate types of homes. See "Confounding Variables" on page 172 for a discussion of confounding variables.



회귀분석 진단하기 > Heteroskedasticity, Non-Normality, and Correlated Errors

Regression Diagnostics

Heteroskedasticity, Non-Normality, and Correlated Errors

Statisticians pay considerable attention to the distribution of the residuals. It turns out that ordinary least squares (see “**Least Squares**” on page 148) are unbiased, and in some cases are the “optimal” estimator, under a wide range of distributional assumptions. This means that in most problems, data scientists do not need to be too concerned with the distribution of the residuals.

The distribution of the residuals is relevant mainly for the validity of formal statistical inference (hypothesis tests and p-values), which is of minimal importance to data scientists concerned mainly with predictive accuracy. Normally distributed errors are a sign that the model is complete; errors that are not normally distributed indicate the model may be missing something. For formal inference to be fully valid, the residuals are assumed to be normally distributed, have the same variance, and be independent. One area where this may be of concern to data scientists is the standard calculation of confidence intervals for predicted values, which are based upon the assumptions about the residuals (see “**Confidence and Prediction Intervals**” on page 161).

Heteroskedasticity is the lack of constant residual variance across the range of the predicted values. In other words, errors are greater for some portions of the range than for others. Visualizing the data is a convenient way to analyze residuals.

이분산성, 비정규성, 그리고 오류의 상관관계

최소제곱법의 특성

- 잔차의 분포에 대한 다양한 가정 하에서도, 최소제곱법(OLS)은 편향되지 않은 추정치를 제공하고, 때로는 '최적'의 추정치를 제공한다.
- 이는 대부분의 문제에서 데이터 과학자들이 잔차의 분포에 대해 지나치게 걱정할 필요가 없음을 의미한다.

잔차의 분포 중요성

- 잔차의 분포는 주로 공식적인 통계적 추론(가설 검정과 p-값)의 유효성과 관련이 있다. 예측 정확도에 주로 관심이 있는 데이터 과학자들에게는 이는 크게 중요하지 않다. 정규분포를 따르는 오류는 모델이 완전하다는 신호이며, 정규분포를 따르지 않는 오류는 모델이 무언가 빠져 있을 수 있음을 나타낸다. 공식적인 추론이 완전히 유효하기 위해서는 잔차가 정규분포를 따르고, 같은 분산을 가지며, 독립적이어야 한다. 예측값에 대한 신뢰 구간 계산과 같이 데이터 과학자에게 중요할 수 있는 영역도 있다.

이분산성(Heteroskedasticity)

- 이분산성은 예측값의 범위에 따라 일정하지 않은 잔차의 분산을 의미
- 즉, 범위의 일부 부분에서는 오류가 다른 부분보다 크다. 따라서, 데이터를 시각화하는 것은 잔차를 분석하는 편리한 방법입니다.

=> 요약하자면, 계량통계학의 측면에서 이분산성, 비정규성, 오류 상관관계의 중요성을 강조하면서도, 데이터 과학자들이 예측 정확도에 주로 집중하는 경우, 이러한 요소들에 대해 너무 심각하게 고민할 필요가 없음을 의미한다.



회귀분석 진단하기 > Heteroskedasticity, Non-Normality, and Correlated Errors

Regression Diagnostics

The following code in R plots the absolute residuals versus the predicted values for the `lm_98105` regression fit in “Outliers” on page 177:

```
df <- data.frame(resid = residuals(lm_98105), pred = predict(lm_98105))
ggplot(df, aes(pred, abs(resid))) + geom_point() + geom_smooth()
```

Figure 4-7 shows the resulting plot. Using `geom_smooth`, it is easy to superpose a smooth of the absolute residuals. The function calls the loess method (locally estimated scatterplot smoothing) to produce a smoothed estimate of the relationship between the variables on the x-axis and y-axis in a scatterplot (see “Scatterplot Smoothers” on page 185).

In *Python*, the *seaborn* package has the `regplot` function to create a similar figure:

```
fig, ax = plt.subplots(figsize=(5, 5))
sns.regplot(result_98105.fittedvalues, np.abs(result_98105.resid),
            scatter_kws={'alpha': 0.25}, line_kws={'color': 'C1'},
            lowess=True, ax=ax)
ax.set_xlabel('predicted')
ax.set_ylabel('abs(residual)')
```

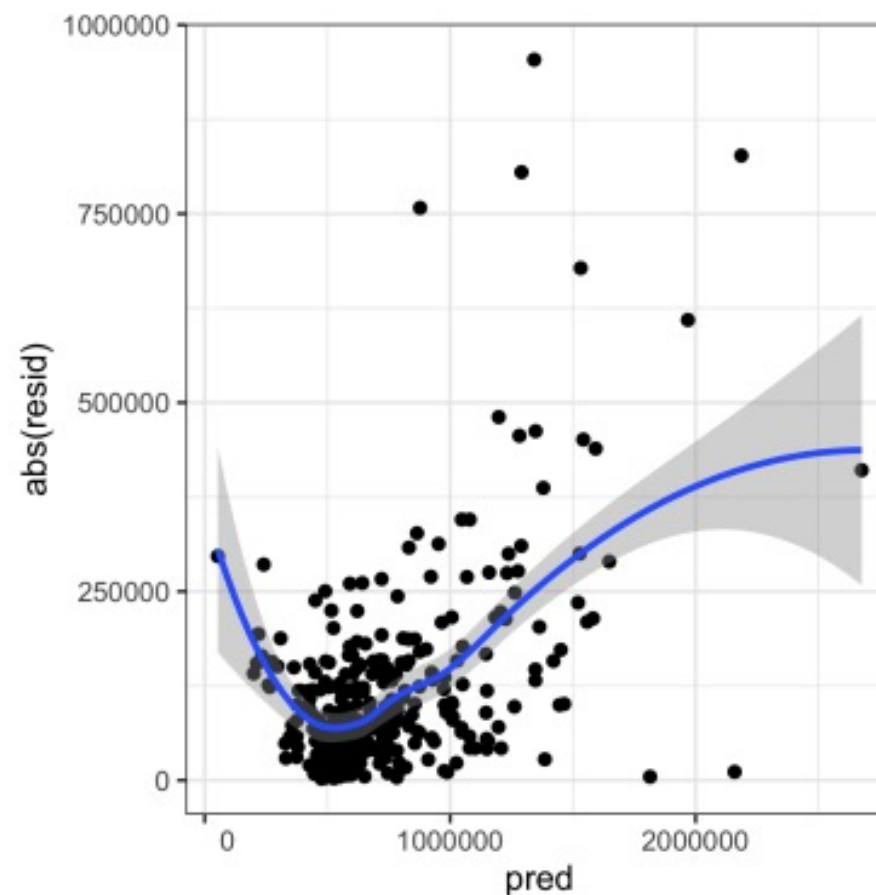


Figure 4-7. A plot of the absolute value of the residuals versus the predicted values



회귀분석 진단하기 > Heteroskedasticity, Non-Normality, and Correlated Errors

Regression Diagnostics

Evidently, the variance of the residuals tends to increase for higher-valued homes but is also large for lower-valued homes. This plot indicates that `lm_98105` has *heteroskedastic* errors.



Why Would a Data Scientist Care About Heteroskedasticity?

Heteroskedasticity indicates that prediction errors differ for different ranges of the predicted value, and may suggest an incomplete model. For example, the heteroskedasticity in `lm_98105` may indicate that the regression has left something unaccounted for in high- and low-range homes.

Figure 4-8 is a histogram of the standardized residuals for the `lm_98105` regression. The distribution has decidedly longer tails than the normal distribution and exhibits mild skewness toward larger residuals.

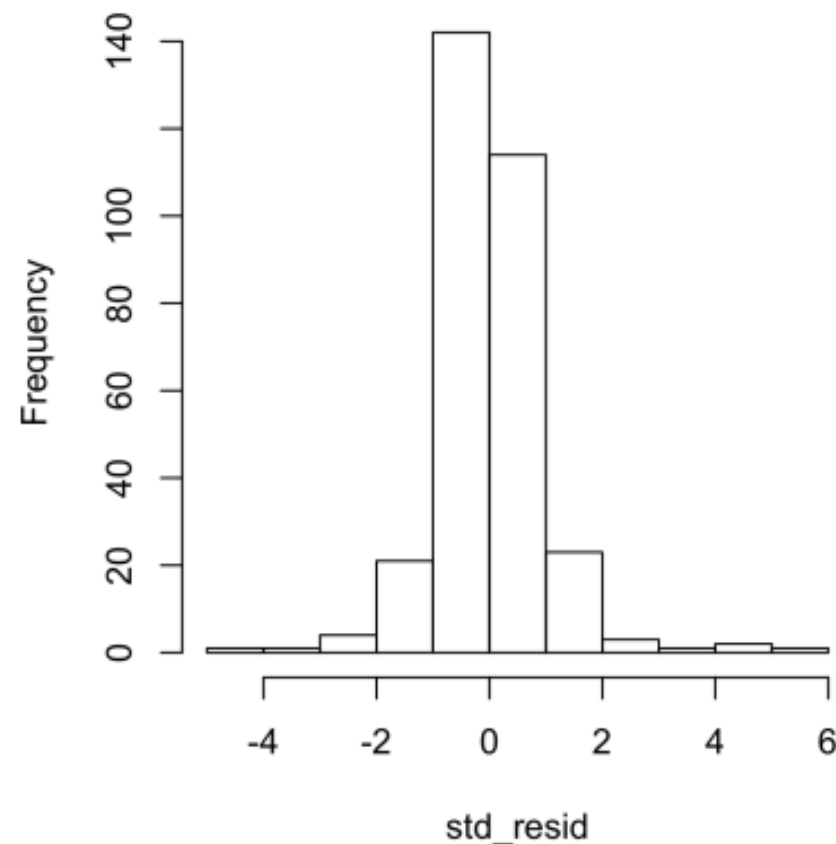


Figure 4-8. A histogram of the residuals from the regression of the housing data



회귀분석 진단하기 > Heteroskedasticity, Non-Normality, and Correlated Errors

Regression Diagnostics

Statisticians may also check the assumption that the errors are independent. This is particularly true for data that is collected over time or space. The *Durbin-Watson* statistic can be used to detect if there is significant autocorrelation in a regression involving time series data. If the errors from a regression model are correlated, then this information can be useful in making short-term forecasts and should be built into the model. See *Practical Time Series Forecasting with R*, 2nd ed., by Galit Shmueli and Kenneth Lichtendahl (Axelrod Schnall, 2018) to learn more about how to build autocorrelation information into regression models for time series data. If longer-term forecasts or explanatory models are the goal, excess autocorrelated data at the microlevel may distract. In that case, smoothing, or less granular collection of data in the first place, may be in order.

Even though a regression may violate one of the distributional assumptions, should we care? Most often in data science, the interest is primarily in predictive accuracy, so some review of heteroskedasticity may be in order. You may discover that there is some signal in the data that your model has not captured. However, satisfying distributional assumptions simply for the sake of validating formal statistical inference (p-values, F-statistics, etc.) is not that important for the data scientist.



Scatterplot Smoothers

Regression is about modeling the relationship between the response and predictor variables. In evaluating a regression model, it is useful to use a *scatterplot smoother* to visually highlight relationships between two variables.

For example, in [Figure 4-7](#), a smooth of the relationship between the absolute residuals and the predicted value shows that the variance of the residuals depends on the value of the residual. In this case, the `loess` function was used; `loess` works by repeatedly fitting a series of local regressions to contiguous subsets to come up with a smooth. While `loess` is probably the most commonly used smoother, other scatterplot smoothers are available in *R*, such as `super smooth (supsmu)` and `kernel smoothing (ksmooth)`. In *Python*, we can find additional smoothers in `scipy` (`wiener` or `sav`) and `statsmodels` (`kernel_regression`). For the purposes of evaluating a regression model, there is typically no need to worry about the details of these scatterplot smooths.



Partial Residual Plots and Nonlinearity

Partial residual plots are a way to visualize how well the estimated fit explains the relationship between a predictor and the outcome. The basic idea of a partial residual plot is to isolate the relationship between a predictor variable and the response, *taking into account all of the other predictor variables*. A partial residual might be thought of as a “synthetic outcome” value, combining the prediction based on a single predictor with the actual residual from the full regression equation. A partial residual for predictor X_i is the ordinary residual plus the regression term associated with X_i :

$$\text{Partial residual} = \text{Residual} + \hat{b}_i X_i$$

where $\hat{b}_i$ is the estimated regression coefficient. The `predict` function in R has an option to return the individual regression terms $\hat{b}_i X_i$:

```
terms <- predict(lm_98105, type='terms')
partial_resid <- resid(lm_98105) + terms
```

The partial residual plot displays the X_i predictor on the x-axis and the partial residuals on the y-axis. Using `ggplot2` makes it easy to superpose a smooth of the partial residuals:

```
df <- data.frame(SqFtTotLiving = house_98105[, 'SqFtTotLiving'],
                 Terms = terms[, 'SqFtTotLiving'],
                 PartialResid = partial_resid[, 'SqFtTotLiving'])
ggplot(df, aes(SqFtTotLiving, PartialResid)) +
  geom_point(shape=1) + scale_shape(solid = FALSE) +
  geom_smooth(linetype=2) +
  geom_line(aes(SqFtTotLiving, Terms))
```

The `statsmodels` package has the method `sm.graphics.plot_ccpr` that creates a similar partial residual plot:

```
sm.graphics.plot_ccpr(result_98105, 'SqFtTotLiving')
```

The R and Python graphs differ by a constant shift. In R, a constant is added so that the mean of the terms is zero.



회귀분석 진단하기 > Partial Residual Plots and Nonlinearity

Regression Diagnostics

The resulting plot is shown in [Figure 4-9](#). The partial residual is an estimate of the contribution that SqFtTotLiving adds to the sales price. The relationship between SqFtTotLiving and the sales price is evidently nonlinear (dashed line). The regression line (solid line) underestimates the sales price for homes less than 1,000 square feet and overestimates the price for homes between 2,000 and 3,000 square feet. There are too few data points above 4,000 square feet to draw conclusions for those homes.

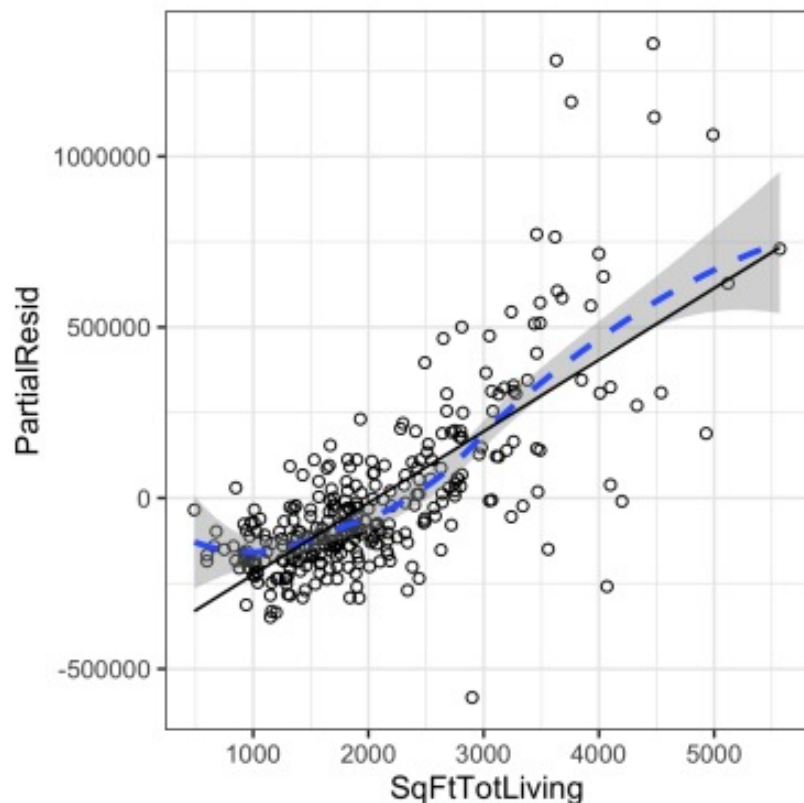


Figure 4-9. A partial residual plot for the variable SqFtTotLiving

This nonlinearity makes sense in this case: adding 500 feet in a small home makes a much bigger difference than adding 500 feet in a large home. This suggests that, instead of a simple linear term for SqFtTotLiving, a nonlinear term should be considered (see [“Polynomial and Spline Regression” on page 187](#)).

Key Ideas

- While outliers can cause problems for small data sets, the primary interest with outliers is to identify problems with the data, or locate anomalies.
- Single records (including regression outliers) can have a big influence on a regression equation with small data, but this effect washes out in big data.
- If the regression model is used for formal inference (p-values and the like), then certain assumptions about the distribution of the residuals should be checked. In general, however, the distribution of residuals is not critical in data science.
- The partial residuals plot can be used to qualitatively assess the fit for each regression term, possibly leading to alternative model specification.



Polynomial and Spline Regression

The relationship between the response and a predictor variable isn't necessarily linear. The response to the dose of a drug is often nonlinear: doubling the dosage generally doesn't lead to a doubled response. The demand for a product isn't a linear function of marketing dollars spent; at some point, demand is likely to be saturated. There are many ways that regression can be extended to capture these nonlinear effects.

Key Terms for Nonlinear Regression

Polynomial regression

Adds polynomial terms (squares, cubes, etc.) to a regression.

Spline regression

Fitting a smooth curve with a series of polynomial segments.

Knots

Values that separate spline segments.

Generalized additive models

Spline models with automated selection of knots.

Synonym

GAM



Nonlinear Regression

When statisticians talk about *nonlinear regression*, they are referring to models that can't be fit using least squares. What kind of models are nonlinear? Essentially all models where the response cannot be expressed as a linear combination of the predictors or some transform of the predictors. Nonlinear regression models are harder and computationally more intensive to fit, since they require numerical optimization. For this reason, it is generally preferred to use a linear model if possible.



회귀분석 (다항/스플라인) > Polynomial

Polynomial and Spline Regression

Polynomial

Polynomial regression involves including polynomial terms in a regression equation. The use of polynomial regression dates back almost to the development of regression itself with a paper by Gergonne in 1815. For example, a quadratic regression between the response Y and the predictor X would take the form:

$$Y = b_0 + b_1X + b_2X^2 + e$$

Polynomial regression can be fit in *R* through the `poly` function. For example, the following fits a quadratic polynomial for `SqFtTotLiving` with the King County housing data:

```
lm(AdjSalePrice ~ poly(SqFtTotLiving, 2) + SqFtLot +
    BldgGrade + Bathrooms + Bedrooms,
    data=house_98105)
```

Call:

```
lm(formula = AdjSalePrice ~ poly(SqFtTotLiving, 2) + SqFtLot +
    BldgGrade + Bathrooms + Bedrooms, data = house_98105)
```

Coefficients:

(Intercept)	poly(SqFtTotLiving, 2)1	poly(SqFtTotLiving, 2)2
-402530.47	3271519.49	776934.02
SqFtLot	BldgGrade	Bathrooms
32.56	135717.06	-1435.12
Bedrooms		
-9191.94		

In *statsmodels*, we add the squared term to the model definition using `I(SqFtTotLiving**2)`:

```
model_poly = smf.ols(formula='AdjSalePrice ~ SqFtTotLiving + ' +
    '+ I(SqFtTotLiving**2) + ' +
    'SqFtLot + Bathrooms + Bedrooms + BldgGrade', data=house_98105)
result_poly = model_poly.fit()
result_poly.summary() ❶
```

- ❶ The intercept and the polynomial coefficients are different compared to *R*. This is due to different implementations. The remaining coefficients and the predictions are equivalent.

There are now two coefficients associated with `SqFtTotLiving`: one for the linear term and one for the quadratic term.

The partial residual plot (see “[Partial Residual Plots and Nonlinearity](#)” on page 185) indicates some curvature in the regression equation associated with `SqFtTotLiving`. The fitted line more closely matches the smooth (see “[Splines](#)” on page 189) of the partial residuals as compared to a linear fit (see [Figure 4-10](#)).



회귀분석 (다항/스플라인) > Polynomial

Polynomial and Spline Regression

The `statsmodels` implementation works only for linear terms. The accompanying source code gives an implementation that will work for polynomial regression as well.

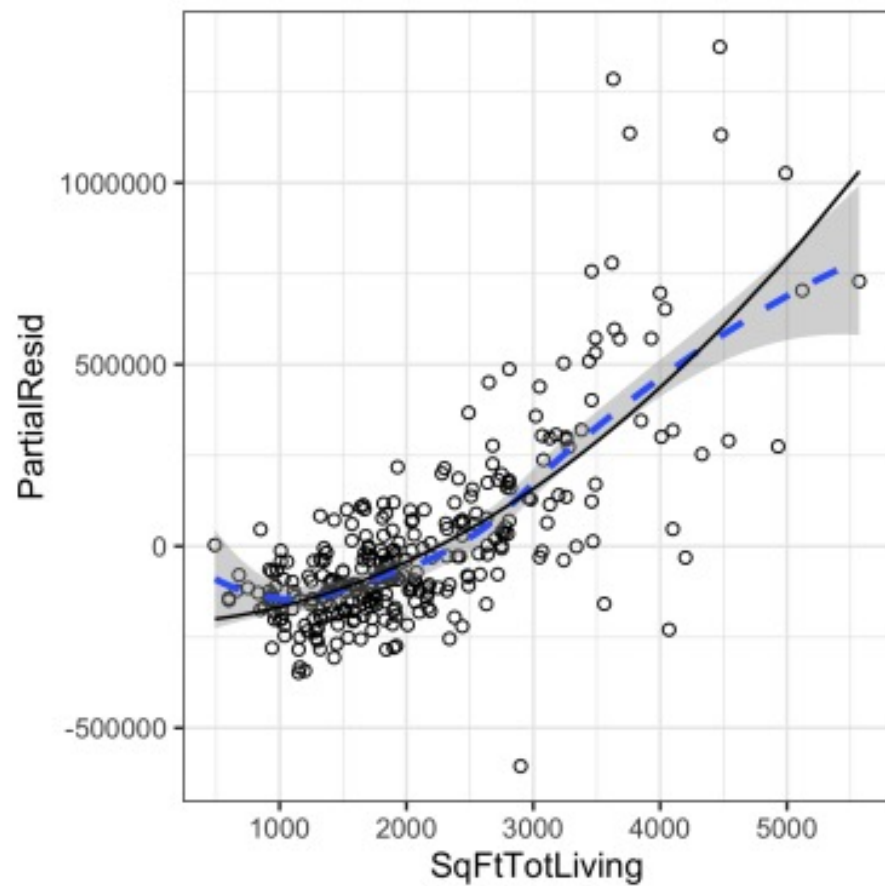


Figure 4-10. A polynomial regression fit for the variable `SqFtTotLiving` (solid line) versus a smooth (dashed line; see the following section about splines)



회귀분석 (다항/스플라인) > Splines

Polynomial and Spline Regression

Splines

Polynomial regression captures only a certain amount of curvature in a nonlinear relationship. Adding in higher-order terms, such as a cubic quartic polynomial, often leads to undesirable “wiggleness” in the regression equation. An alternative, and often superior, approach to modeling nonlinear relationships is to use *splines*. *Splines* provide a way to smoothly interpolate between fixed points. Splines were originally used by draftsmen to draw a smooth curve, particularly in ship and aircraft building.

The splines were created by bending a thin piece of wood using weights, referred to as “ducks”; see Figure 4-11.



Figure 4-11. Splines were originally created using bendable wood and “ducks” and were used as a draftsman’s tool to fit curves (photo courtesy of Bob Perry)

The technical definition of a spline is a series of piecewise continuous polynomials. They were first developed during World War II at the US Aberdeen Proving Grounds by I. J. Schoenberg, a Romanian mathematician. The polynomial pieces are smoothly connected at a series of fixed points in a predictor variable, referred to as *knots*. Formulation of splines is much more complicated than polynomial regression; statistical software usually handles the details of fitting a spline. The R package *splines* includes the function `bs` to create a *b-spline* (basis spline) term in a regression model. For example, the following adds a b-spline term to the house regression model:

```
library(splines)
knots <- quantile(house_98105$SqFtTotLiving, p=c(.25, .5, .75))
lm_spline <- lm(AdjSalePrice ~ bs(SqFtTotLiving, knots=knots, degree=3) +
  SqFtLot + Bathrooms + Bedrooms + BldgGrade, data=house_98105)
```

Two parameters need to be specified: the degree of the polynomial and the location of the knots. In this case, the predictor `SqFtTotLiving` is included in the model using a cubic spline (`degree=3`). By default, `bs` places knots at the boundaries; in addition, knots were also placed at the lower quartile, the median quartile, and the upper quartile.



회귀분석 (다항/스플라인) > Splines

Polynomial and Spline Regression

The `statsmodels` formula interface supports the use of splines in a similar way to *R*. Here, we specify the *b-spline* using `df`, the degrees of freedom. This will create `df - degree = 6 - 3 = 3` internal knots with positions calculated in the same way as in the *R* code above:

```
formula = 'AdjSalePrice ~ bs(SqFtTotLiving, df=6, degree=3) + ' +  
          'SqFtLot + Bathrooms + Bedrooms + BldgGrade'  
model_spline = smf.ols(formula=formula, data=house_98105)  
result_spline = model_spline.fit()
```

In contrast to a linear term, for which the coefficient has a direct meaning, the coefficients for a spline term are not interpretable. Instead, it is more useful to use the visual display to reveal the nature of the spline fit. **Figure 4-12** displays the partial residual plot from the regression. In contrast to the polynomial model, the spline model more closely matches the smooth, demonstrating the greater flexibility of splines. In this case, the line more closely fits the data. Does this mean the spline regression is a better model? Not necessarily: it doesn't make economic sense that very small homes (less than 1,000 square feet) would have higher value than slightly larger homes. This is possibly an artifact of a confounding variable; see **"Confounding Variables"** on page 172.

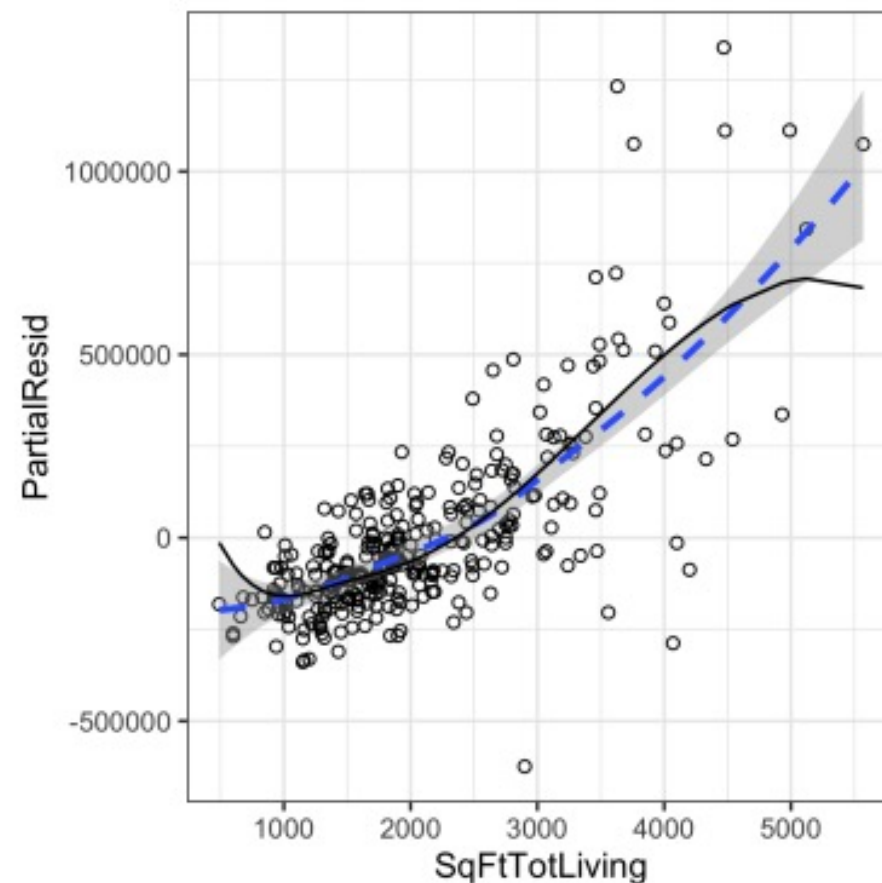


Figure 4-12. A spline regression fit for the variable `SqFtTotLiving` (solid line) compared to a smooth (dashed line)



회귀분석 (다항/스플라인) > Generalized Additive Models

Polynomial and Spline Regression

Generalized Additive Models

Suppose you suspect a nonlinear relationship between the response and a predictor variable, either by a priori knowledge or by examining the regression diagnostics. Polynomial terms may not be flexible enough to capture the relationship, and spline terms require specifying the knots. *Generalized additive models*, or GAM, are a flexible modeling technique that can be used to automatically fit a spline regression. The `mgcv` package in R can be used to fit a GAM model to the housing data:

```
library(mgcv)
lm_gam <- gam(AdjSalePrice ~ s(SqFtTotLiving) + SqFtLot +
              Bathrooms + Bedrooms + BldgGrade,
              data=house_98105)
```

The term `s(SqFtTotLiving)` tells the `gam` function to find the “best” knots for a spline term (see Figure 4-13).

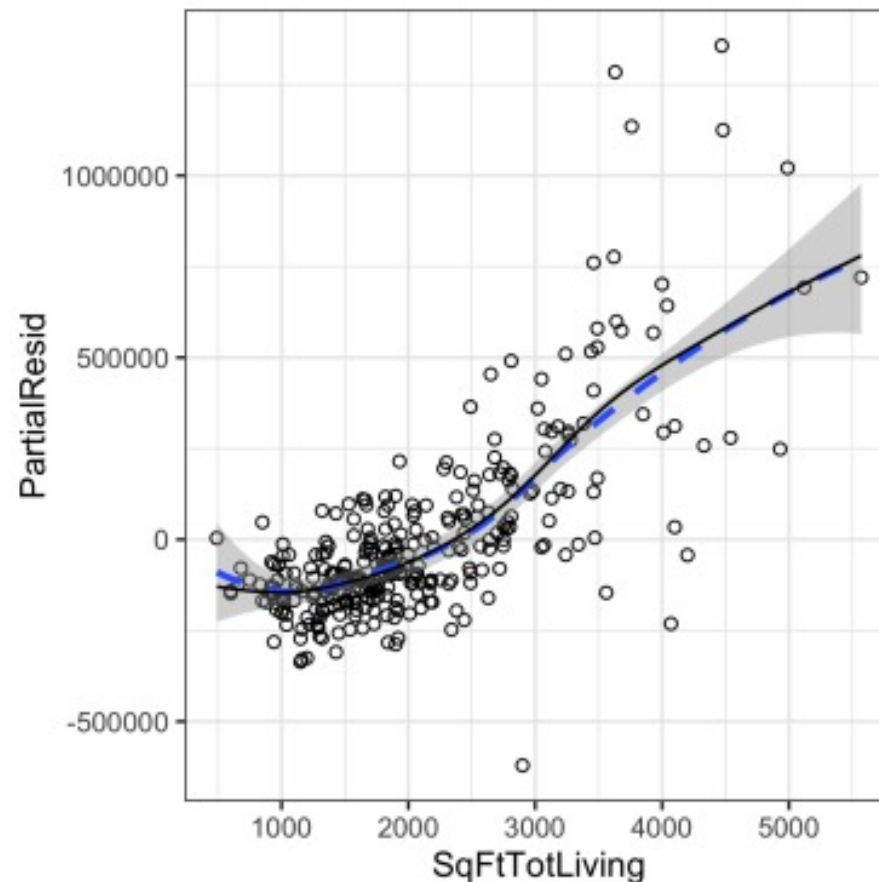


Figure 4-13. A GAM regression fit for the variable `SqFtTotLiving` (solid line) compared to a smooth (dashed line)



회귀분석 (다항/스플라인) > Generalized Additive Models

Polynomial and Spline Regression

In *Python*, we can use the *pyGAM* package. It provides methods for regression and classification. Here, we use *LinearGAM* to create a regression model:

```
predictors = ['SqFtTotLiving', 'SqFtLot', 'Bathrooms', 'Bedrooms', 'BldgGrade']
outcome = 'AdjSalePrice'
X = house_98105[predictors].values
y = house_98105[outcome]
```

```
gam = LinearGAM(s(0, n_splines=12) + l(1) + l(2) + l(3) + l(4)) ❶
gam.gridsearch(X, y)
```

- ❶ The default value for *n_splines* is 20. This leads to overfitting for larger *SqFtTotLiving* values. A value of 12 leads to a more reasonable fit.

Key Ideas

- Outliers in a regression are records with a large residual.
- Multicollinearity can cause numerical instability in fitting the regression equation.
- A confounding variable is an important predictor that is omitted from a model and can lead to a regression equation with spurious relationships.
- An interaction term between two variables is needed if the effect of one variable depends on the level or magnitude of the other.
- Polynomial regression can fit nonlinear relationships between predictors and the outcome variable.
- Splines are series of polynomial segments strung together, joining at knots.
- We can automate the process of specifying the knots in splines using generalized additive models (GAM).

Q&A and Break Time

질의응답 및 휴식시간 (10분)



2부: 분류

- 1) Naïve Bayes
- 2) 분류분석
- 3) 로지스틱 회귀





비즈니스 문제에 대한 의사 결정을 자동화시 필요한 분류 모델

예시) 이메일이 피싱인지? 고객이 이탈할 가능성이 있는지? 웹 사용자가 광고를 클릭할 가능성이 있는지?

분류란?

- 결과가 알려진 데이터에 대해 모델을 먼저 훈련시킨 다음 결과가 알려지지 않은 데이터에 모델을 적용하는 지도 학습의 한 형태
- 아마도 가장 중요한 예측 형식
- 목표는 레코드가 1인지 0인지(피싱/피싱 아님, 클릭/클릭하지 않음, 이탈/제거하지 않음) 또는 경우에 따라 예측하는 것
- 여러 범주 중 하나(예: 받은편지함을 '기본', '소셜', '프로모션' 또는 '포럼'으로 필터링하는 Gmail)와 같이 종종 단순한 이진 분류 이상의 것 또한 필요
- 클래스에 속할 예측 확률을 알 수도 있다
- 모델이 단순히 이진 분류를 할당하도록 하는 대신 대부분의 알고리즘은 관심 클래스에 속하는 확률 점수를 반환할 수 있다



Python의 scikit-learn에서 로지스틱 회귀는 대부분의 분류 방법과 마찬가지로 두 가지 예측 방법을 제공

- predict(클래스 반환) 및 predict_proba(각 클래스의 확률 반환)
- 슬라이딩 컷오프를 사용하여 확률값을 클래스로 변환할 수 있다

일반적인 접근 방식

1. 관심 클래스에 대한 컷오프 확률을 설정 (그 이상에서는 레코드가 해당 클래스에 속하는 것으로 간주)
2. 레코드가 관심 클래스에 속할 확률을 추정 (모든 모델로)
3. 해당 확률이 컷오프 확률보다 높으면 관심 클래스에 새 레코드를 할당
4. 컷오프가 높을수록 1, 즉 관심 클래스에 속하는 것으로 예측되는 레코드가 적어짐
컷오프가 낮을수록 더 많은 레코드가 1로 예측

이번 수업에서는 확률값을 분류하고 추정하는 몇 가지 핵심 기술을 다룰 예정



Naive Bayes

The naive Bayes algorithm uses the probability of observing predictor values, given an outcome, to estimate what is really of interest: the probability of observing outcome $Y = i$, given a set of predictor values.¹

Key Terms for Naive Bayes

Conditional probability

The probability of observing some event (say, $X = i$) given some other event (say, $Y = j$), written as $P(X_i | Y_j)$.

Posterior probability

The probability of an outcome after the predictor information has been incorporated (in contrast to the *prior probability* of outcomes, not taking predictor information into account).

¹ This and subsequent sections in this chapter © 2020 Datastats, LLC, Peter Bruce, Andrew Bruce, and Peter Gedeck; used with permission.

To understand naive Bayesian classification, we can start out by imagining complete or exact Bayesian classification. For each record to be classified:

1. Find all the other records with the same predictor profile (i.e., where the predictor values are the same).
2. Determine what classes those records belong to and which class is most prevalent (i.e., probable).
3. Assign that class to the new record.

The preceding approach amounts to finding all the records in the sample that are exactly like the new record to be classified in the sense that all the predictor values are identical.



Predictor variables must be categorical (factor) variables in the standard naive Bayes algorithm. See ["Numeric Predictor Variables" on page 200](#) for two workarounds for using continuous variables.



Why Exact Bayesian Classification Is Impractical

When the number of predictor variables exceeds a handful, many of the records to be classified will be without exact matches. Consider a model to predict voting on the basis of demographic variables. Even a sizable sample may not contain even a single match for a new record who is a male Hispanic with high income from the US Midwest who voted in the last election, did not vote in the prior election, has three daughters and one son, and is divorced. And this is with just eight variables, a small number for most classification problems. The addition of just a single new variable with five equally frequent categories reduces the probability of a match by a factor of 5.

베이지안 분류가 비현실적인 이유

- 예측 변수가 몇 개만 넘어가도 정확한 일치를 찾는 것이 어렵기 때문
- 예를 들어, 인구 통계 변수를 기반으로 투표를 예측하는 모델을 생각해 보자. 상당한 크기의 샘플조차도 새로운 기록에 대해 정확하게 맞추지 못할 수도 있다.
- 또한, 미국 중서부 출신의 고소득 히스패닉 남성으로서 지난 선거에는 투표했지만 그 이전 선거에는 투표하지 않았으며, 세 딸과 한 아들을 둔 이혼남인 경우는 그 확률이 드물기 때문이다.
 - 이것은 단지 8개의 변수에 불과하지만, 대부분의 분류 문제에서는 이보다 훨씬 많은 변수를 다룹니다. 단 하나의 새로운 변수가 추가되어도 그 변수가 다섯 개의 균등한 범주를 갖는다면 정확하게 맞출 확률이 5분의 1로 줄어듭니다.



The Naive Solution

In the naive Bayes solution, we no longer restrict the probability calculation to those records that match the record to be classified. Instead, we use the entire data set. The naive Bayes modification is as follows:

1. For a binary response $Y = i$ ($i = 0$ or 1), estimate the individual conditional probabilities for each predictor $P(X_j | Y = i)$; these are the probabilities that the predictor value is in the record when we observe $Y = i$. This probability is estimated by the proportion of X_j values among the $Y = i$ records in the training set.
2. Multiply these probabilities by each other, and then by the proportion of records belonging to $Y = i$.
3. Repeat steps 1 and 2 for all the classes.
4. Estimate a probability for outcome i by taking the value calculated in step 2 for class i and dividing it by the sum of such values for all classes.
5. Assign the record to the class with the highest probability for this set of predictor values.

This naive Bayes algorithm can also be stated as an equation for the probability of observing outcome $Y = i$, given a set of predictor values $X_1, \dots, X_p$:

$$P(Y = i | X_1, X_2, \dots, X_p)$$

Here is the full formula for calculating class probabilities using exact Bayes classification:

$$P(Y = i | X_1, X_2, \dots, X_p) = \frac{P(Y = i)P(X_1, \dots, X_p | Y = i)}{P(Y = 0)P(X_1, \dots, X_p | Y = 0) + P(Y = 1)P(X_1, \dots, X_p | Y = 1)}$$

Under the naive Bayes assumption of conditional independence, this equation changes into:

$$\begin{aligned} P(Y = i | X_1, X_2, \dots, X_p) \\ = \frac{P(Y = i)P(X_1 | Y = i) \dots P(X_p | Y = i)}{P(Y = 0)P(X_1 | Y = 0) \dots P(X_p | Y = 0) + P(Y = 1)P(X_1 | Y = 1) \dots P(X_p | Y = 1)} \end{aligned}$$



나이프 베이즈 > The Naive Solution

Naive Bayes

Why is this formula called “naive”? We have made a simplifying assumption that the *exact conditional probability* of a vector of predictor values, given observing an outcome, is sufficiently well estimated by the product of the individual conditional probabilities $P(X_j|Y=i)$. In other words, in estimating $P(X_j|Y=i)$ instead of $P(X_1, X_2, \dots, X_p|Y=i)$, we are assuming X_j is *independent* of all the other predictor variables X_k for $k \neq j$.

Several packages in R can be used to estimate a naive Bayes model. The following fits a model to the loan payment data using the `klaR` package:

```
library(klaR)
naive_model <- NaiveBayes(outcome ~ purpose_ + home_ + emp_len_,
                          data = na.omit(loan_data))

naive_model$table
$purpose_
      var
grouping credit_card debt_consolidation home_improvement major_purchase
paid off  0.18759649          0.55215915          0.07150104          0.05359270
default   0.15151515          0.57571347          0.05981209          0.03727229

      var
grouping      medical      other small_business
paid off 0.01424728 0.09990737          0.02099599
default  0.01433549 0.11561025          0.04574126

$home_
      var
grouping MORTGAGE      OWN      RENT
paid off 0.4894800 0.0808963 0.4296237
default  0.4313440 0.0832782 0.4853778

$emp_len_
      var
grouping < 1 Year > 1 Year
paid off 0.03105289 0.96894711
default  0.04728508 0.95271492
```

The output from the model is the conditional probabilities $P(X_j|Y=i)$.



나이프 베이즈 > The Naive Solution

Naive Bayes

In *Python* we can use `sklearn.naive_bayes.MultinomialNB` from `scikit-learn`. We need to convert the categorical features to dummy variables before we fit the model:

```

predictors = ['purpose_', 'home_', 'emp_len_']
outcome = 'outcome'
X = pd.get_dummies(loan_data[predictors], prefix='', prefix_sep='')
y = loan_data[outcome]

naive_model = MultinomialNB(alpha=0.01, fit_prior=True)
naive_model.fit(X, y)

```

It is possible to derive the conditional probabilities from the fitted model using the property `feature_log_prob_`.

The model can be used to predict the outcome of a new loan. We use the last value of the data set for testing:

```

new_loan <- loan_data[147, c('purpose_', 'home_', 'emp_len_')]
row.names(new_loan) <- NULL
new_loan
  purpose_ home_ emp_len_
1 small_business MORTGAGE > 1 Year

```

In *Python*, we get this value as follows:

```
new_loan = X.loc[146:146, :]
```

In this case, the model predicts a default (R):

```

predict(naive_model, new_loan)
$class
[1] default
Levels: paid off default

$posterior
      paid off   default
[1,] 0.3463013 0.6536987

```

As we discussed, `scikit-learn`'s classification models have two methods—`predict`, which returns the predicted class, and `predict_proba`, which returns the class probabilities:

```

print('predicted class: ', naive_model.predict(new_loan)[0])

probabilities = pd.DataFrame(naive_model.predict_proba(new_loan),
                             columns=loan_data[outcome].cat.categories)
print('predicted probabilities', probabilities)
--
predicted class: default
predicted probabilities
      default paid off
0 0.653696 0.346304

```

The prediction also returns a posterior estimate of the probability of default. The naive Bayesian classifier is known to produce *biased* estimates. However, where the goal is to *rank* records according to the probability that $Y = 1$, unbiased estimates of probability are not needed, and naive Bayes produces good results.



Numeric Predictor Variables

The Bayesian classifier works only with categorical predictors (e.g., with spam classification, where the presence or absence of words, phrases, characters, and so on lies at the heart of the predictive task). To apply naive Bayes to numerical predictors, one of two approaches must be taken:

- Bin and convert the numerical predictors to categorical predictors and apply the algorithm of the previous section.
- Use a probability model—for example, the normal distribution (see “**Normal Distribution**” on page 69)—to estimate the conditional probability $P(X_j | Y = i)$.



When a predictor category is absent in the training data, the algorithm assigns *zero probability* to the outcome variable in new data, rather than simply ignoring this variable and using the information from other variables, as other methods might. Most implementations of Naive Bayes use a smoothing parameter (Laplace Smoothing) to prevent this.

Key Ideas

- Naive Bayes works with categorical (factor) predictors and outcomes.
- It asks, “Within each outcome category, which predictor categories are most probable?”
- That information is then inverted to estimate probabilities of outcome categories, given predictor values.

Discriminant Analysis

Discriminant analysis is the earliest statistical classifier; it was introduced by R. A. Fisher in 1936 in an article published in the *Annals of Eugenics* journal.²

Key Terms for Discriminant Analysis

Covariance

A measure of the extent to which one variable varies in concert with another (i.e., similar magnitude and direction).

Discriminant function

The function that, when applied to the predictor variables, maximizes the separation of the classes.

Discriminant weights

The scores that result from the application of the discriminant function and are used to estimate probabilities of belonging to one class or another.

While discriminant analysis encompasses several techniques, the most commonly used is *linear discriminant analysis*, or *LDA*. The original method proposed by Fisher was actually slightly different from LDA, but the mechanics are essentially the same. LDA is now less widely used with the advent of more sophisticated techniques, such as tree models and logistic regression.

However, you may still encounter LDA in some applications, and it has links to other more widely used methods (such as principal components analysis; see “[Principal Components Analysis](#)” on page 284).



Linear discriminant analysis should not be confused with Latent Dirichlet Allocation, also referred to as LDA. Latent Dirichlet Allocation is used in text and natural language processing and is unrelated to linear discriminant analysis.

² It is certainly surprising that the first article on statistical classification was published in a journal devoted to eugenics. Indeed, there is a [disconcerting connection between the early development of statistics and eugenics](#).



Covariance Matrix

To understand discriminant analysis, it is first necessary to introduce the concept of *covariance* between two or more variables. The covariance measures the relationship between two variables x and z . Denote the mean for each variable by $\bar{x}$ and $\bar{z}$ (see “Mean” on page 9). The covariance $s_{x,z}$ between x and z is given by:

$$s_{x,z} = \frac{\sum_{i=1}^n (x_i - \bar{x})(z_i - \bar{z})}{n - 1}$$

where n is the number of records (note that we divide by $n - 1$ instead of n ; see “Degrees of Freedom, and n or $n - 1$?” on page 15).

As with the correlation coefficient (see “Correlation” on page 30), positive values indicate a positive relationship and negative values indicate a negative relationship. Correlation, however, is constrained to be between -1 and 1 , whereas covariance scale depends on the scale of the variables x and z . The *covariance matrix* Σ for x and z consists of the individual variable variances, s_x^2 and s_z^2 , on the diagonal (where row and column are the same variable) and the covariances between variable pairs on the off-diagonals:

$$\hat{\Sigma} = \begin{bmatrix} s_x^2 & s_{x,z} \\ s_{z,x} & s_z^2 \end{bmatrix}$$



Recall that the standard deviation is used to normalize a variable to a z -score; the covariance matrix is used in a multivariate extension of this standardization process. This is known as Mahalanobis distance (see “Other Distance Metrics” on page 242) and is related to the LDA function.



Fisher's Linear Discriminant

For simplicity, let's focus on a classification problem in which we want to predict a binary outcome y using just two continuous numeric variables (x, z) . Technically, discriminant analysis assumes the predictor variables are normally distributed continuous variables, but, in practice, the method works well even for nonextreme departures from normality, and for binary predictors. Fisher's linear discriminant distinguishes variation *between* groups, on the one hand, from variation *within* groups on the other. Specifically, seeking to divide the records into two groups, linear discriminant analysis (LDA) focuses on maximizing the "between" sum of squares SS_{between} (measuring the variation between the two groups) relative to the "within" sum of squares SS_{within} (measuring the within-group variation). In this case, the two groups correspond to the records (x_0, z_0) for which $y = 0$ and the records (x_1, z_1) for which $y = 1$. The method finds the linear combination $w_x x + w_z z$ that maximizes that sum of squares ratio:

$$\frac{SS_{\text{between}}}{SS_{\text{within}}}$$

The between sum of squares is the squared distance between the two group means, and the within sum of squares is the spread around the means within each group, weighted by the covariance matrix. Intuitively, by maximizing the between sum of squares and minimizing the within sum of squares, this method yields the greatest separation between the two groups.



분류분석 > A Simple Example

Discriminant Analysis

A Simple Example

The MASS package, associated with the book *Modern Applied Statistics with S* by W. N. Venables and B. D. Ripley (Springer, 1994), provides a function for LDA with R. The following applies this function to a sample of loan data using two predictor variables, `borrower_score` and `payment_inc_ratio`, and prints out the estimated linear discriminator weights:

```
library(MASS)
loan_lda <- lda(outcome ~ borrower_score + payment_inc_ratio,
               data=loan3000)

loan_lda$scaling

              LD1
borrower_score    7.17583880
payment_inc_ratio -0.09967559
```

In *Python*, we can use `LinearDiscriminantAnalysis` from `sklearn.discriminant_analysis`. The `scalings_` property gives the estimated weights:

```
loan3000.outcome = loan3000.outcome.astype('category')

predictors = ['borrower_score', 'payment_inc_ratio']
outcome = 'outcome'

X = loan3000[predictors]
y = loan3000[outcome]

loan_lda = LinearDiscriminantAnalysis()
loan_lda.fit(X, y)
pd.DataFrame(loan_lda.scalings_, index=X.columns)
```



Using Discriminant Analysis for Feature Selection

If the predictor variables are normalized prior to running LDA, the discriminator weights are measures of variable importance, thus providing a computationally efficient method of feature selection.



분류분석 > A Simple Example

Discriminant Analysis

The `lda` function can predict the probability of “default” versus “paid off”:

```
pred <- predict(loan_lda)
head(pred$posterior)
  paid off  default
1 0.4464563 0.5535437
2 0.4418466 0.5589534
3 0.7273038 0.2726962
4 0.4937462 0.5062538
5 0.3900475 0.6099525
6 0.5892594 0.4107406
```

The `predict_proba` method of the fitted model returns the probabilities for the “default” and “paid off” outcomes:

```
pred = pd.DataFrame(loan_lda.predict_proba(loan3000[predictors]),
                    columns=loan_lda.classes_)
pred.head()
```

A plot of the predictions helps illustrate how LDA works. Using the output from the `predict` function, a plot of the estimated probability of default is produced as follows:

```
center <- 0.5 * (loan_lda$mean[1, ] + loan_lda$mean[2, ])
slope <- -loan_lda$scaling[1] / loan_lda$scaling[2]
intercept <- center[2] - center[1] * slope

ggplot(data=lda_df, aes(x=borrower_score, y=payment_inc_ratio,
                       color=prob_default)) +
  geom_point(alpha=.6) +
  scale_color_gradientn(colors=c('#ca0020', '#f7f7f7', '#0571b0')) +
  scale_x_continuous(expand=c(0,0)) +
  scale_y_continuous(expand=c(0,0), lim=c(0, 20)) +
  geom_abline(slope=slope, intercept=intercept, color='darkgreen')
```

A similar graph is created in Python using this code:

```
# Use scalings and center of means to determine decision boundary
center = np.mean(loan_lda.means_, axis=0)
slope = - loan_lda.scalings[0] / loan_lda.scalings[1]
intercept = center[1] - center[0] * slope

# payment_inc_ratio for borrower_score of 0 and 20
x_0 = (0 - intercept) / slope
x_20 = (20 - intercept) / slope

lda_df = pd.concat([loan3000, pred['default']], axis=1)
lda_df.head()

fig, ax = plt.subplots(figsize=(4, 4))
g = sns.scatterplot(x='borrower_score', y='payment_inc_ratio',
                   hue='default', data=lda_df,
                   palette=sns.diverging_palette(240, 10, n=9, as_cmap=True),
                   ax=ax, legend=False)

ax.set_ylim(0, 20)
ax.set_xlim(0.15, 0.8)
ax.plot((x_0, x_20), (0, 20), linewidth=3)
ax.plot(*loan_lda.means_.transpose())
```

The resulting plot is shown in **Figure 5-1**. Data points on the left of the diagonal line are predicted to default (probability greater than 0.5).



분류분석 > A Simple Example

Discriminant Analysis

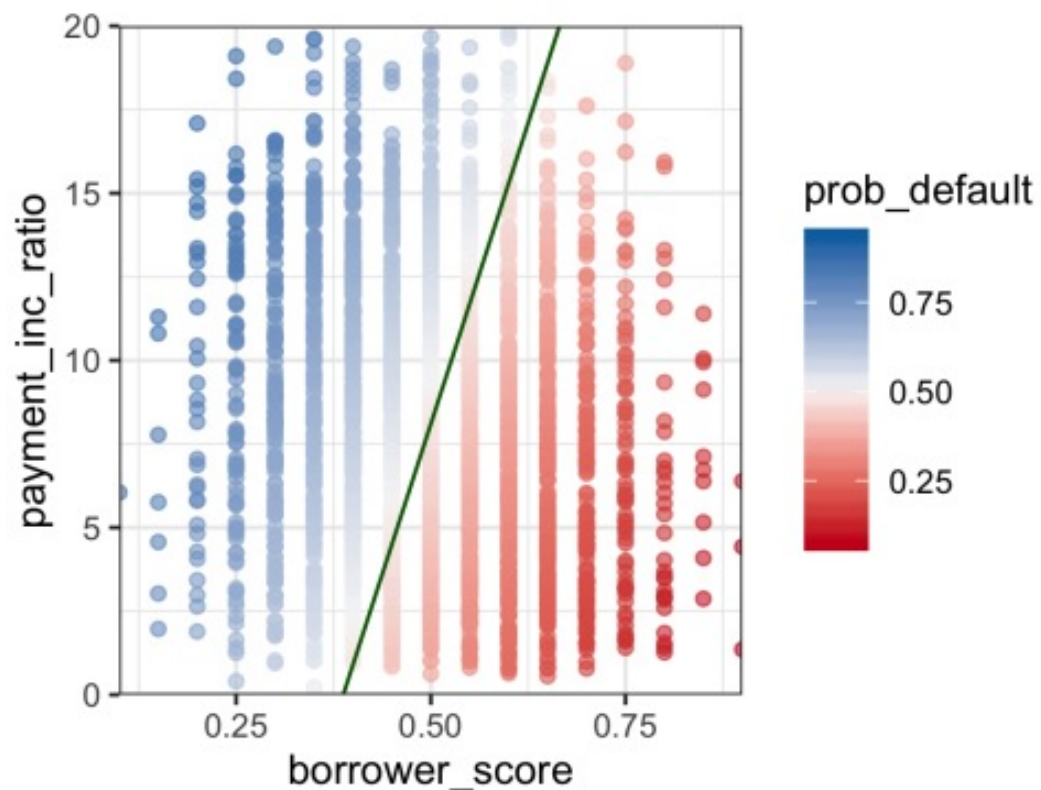


Figure 5-1. LDA prediction of loan default using two variables: a score of the borrower's creditworthiness and the payment-to-income ratio

Using the discriminant function weights, LDA splits the predictor space into two regions, as shown by the solid line. The predictions farther away from the line in both directions have a higher level of confidence (i.e., a probability further away from 0.5).



Extensions of Discriminant Analysis

More predictor variables: while the text and example in this section used just two predictor variables, LDA works just as well with more than two predictor variables. The only limiting factor is the number of records (estimating the covariance matrix requires a sufficient number of records per variable, which is typically not an issue in data science applications).

There are other variants of discriminant analysis. The best known is quadratic discriminant analysis (QDA). Despite its name, QDA is still a linear discriminant function. The main difference is that in LDA, the covariance matrix is assumed to be the same for the two groups corresponding to $Y = 0$ and $Y = 1$. In QDA, the covariance matrix is allowed to be different for the two groups. In practice, the difference in most applications is not critical.

Key Ideas

- Discriminant analysis works with continuous or categorical predictors, as well as with categorical outcomes.
- Using the covariance matrix, it calculates a *linear discriminant function*, which is used to distinguish records belonging to one class from those belonging to another.
- This function is applied to the records to derive weights, or scores, for each record (one weight for each possible class), which determines its estimated class.



Logistic Regression

Logistic regression is analogous to multiple linear regression (see [Chapter 4](#)), except the outcome is binary. Various transformations are employed to convert the problem to one in which a linear model can be fit. Like discriminant analysis, and unlike *K*-Nearest Neighbor and naive Bayes, logistic regression is a structured model approach rather than a data-centric approach. Due to its fast computational speed and its output of a model that lends itself to rapid scoring of new data, it is a popular method.

Key Terms for Logistic Regression

Logit

The function that maps class membership probability to a range from $\pm \infty$ (instead of 0 to 1).

Synonym

Log odds (see below)

Odds

The ratio of “success” (1) to “not success” (0).

Log odds

The response in the transformed model (now linear), which gets mapped back to a probability.



Logistic Response Function and Logit

The key ingredients for logistic regression are the *logistic response function* and the *logit*, in which we map a probability (which is on a 0–1 scale) to a more expansive scale suitable for linear modeling.

The first step is to think of the outcome variable not as a binary label but as the probability p that the label is a “1.” Naively, we might be tempted to model p as a linear function of the predictor variables:

$$p = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q$$

However, fitting this model does not ensure that p will end up between 0 and 1, as a probability must.

Instead, we model p by applying a *logistic response* or *inverse logit* function to the predictors:

$$p = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q)}}$$

This transform ensures that the p stays between 0 and 1.

To get the exponential expression out of the denominator, we consider *odds* instead of probabilities. Odds, familiar to bettors everywhere, are the ratio of “successes” (1) to “nonsuccesses” (0). In terms of probabilities, odds are the probability of an event divided by the probability that the event will not occur. For example, if the probability that a horse will win is 0.5, the probability of “won’t win” is $(1 - 0.5) = 0.5$, and the odds are 1.0:

$$\text{Odds}(Y = 1) = \frac{p}{1 - p}$$

We can obtain the probability from the odds using the inverse odds function:

$$p = \frac{\text{Odds}}{1 + \text{Odds}}$$

We combine this with the logistic response function, shown earlier, to get:

$$\text{Odds}(Y = 1) = e^{\beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q}$$

Finally, taking the logarithm of both sides, we get an expression that involves a linear function of the predictors:

$$\log(\text{Odds}(Y = 1)) = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_q x_q$$

The *log-odds* function, also known as the *logit* function, maps the probability p from $(0, 1)$ to any value $(-\infty, +\infty)$ —see [Figure 5-2](#). The transformation circle is complete; we have used a linear model to predict a probability, which we can in turn map to a class label by applying a cutoff rule—any record with a probability greater than the cutoff is classified as a 1.



로지스틱 회귀 > Logistic Response Function and Logit

Logistic Regression

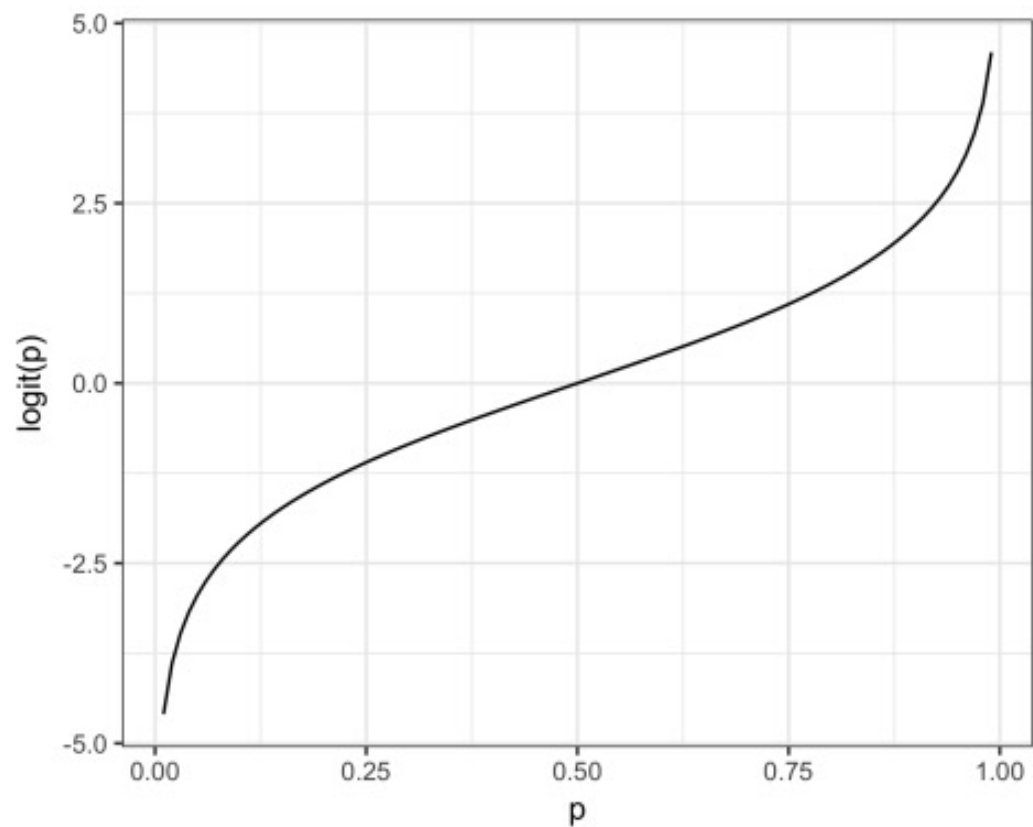


Figure 5-2. Graph of the logit function that maps a probability to a scale suitable for a linear model

Logistic Regression and the GLM

The response in the logistic regression formula is the log odds of a binary outcome of 1. We observe only the binary outcome, not the log odds, so special statistical methods are needed to fit the equation. Logistic regression is a special instance of a *generalized linear model* (GLM) developed to extend linear regression to other settings.

In R, to fit a logistic regression, the `glm` function is used with the `family` parameter set to `binomial`. The following code fits a logistic regression to the personal loan data introduced in “K-Nearest Neighbors” on page 238:

```
logistic_model <- glm(outcome ~ payment_inc_ratio + purpose_ +
                      home_ + emp_len_ + borrower_score,
                      data=loan_data, family='binomial')

logistic_model
```

```
Call: glm(formula = outcome ~ payment_inc_ratio + purpose_ + home_ +
emp_len_ + borrower_score, family = "binomial", data = loan_data)
```

Coefficients:

(Intercept)	payment_inc_ratio
1.63809	0.07974
purpose_debt_consolidation	purpose_home_improvement
0.24937	0.40774
purpose_major_purchase	purpose_medical
0.22963	0.51048
purpose_other	purpose_small_business
0.62066	1.21526
home_OWN	home_RENT
0.04833	0.15732
emp_len_ > 1 Year	borrower_score
-0.35673	-4.61264

```
Degrees of Freedom: 45341 Total (i.e. Null); 45330 Residual
Null Deviance: 62860
Residual Deviance: 57510 AIC: 57540
```

The response is `outcome`, which takes a 0 if the loan is paid off and a 1 if the loan defaults. `purpose_` and `home_` are factor variables representing the purpose of the loan and the home ownership status. As in linear regression, a factor variable with P levels is represented with $P - 1$ columns. By default in R, the *reference* coding is used, and the levels are all compared to the reference level (see “Factor Variables in Regression” on page 163). The reference levels for these factors are `credit_card` and `MORTGAGE`, respectively. The variable `borrower_score` is a score from 0 to 1 representing the creditworthiness of the borrower (from poor to excellent). This variable was created from several other variables using K-Nearest Neighbor—see “KNN as a Feature Engine” on page 247.

In *Python*, we use the `scikit-learn` class `LogisticRegression` from `sklearn.linear_model`. The arguments `penalty` and `C` are used to prevent overfitting by L1 or L2 regularization. Regularization is switched on by default. In order to fit without regularization, we set `C` to a very large value. The `solver` argument selects the used minimizer; the method `liblinear` is the default:

```
predictors = ['payment_inc_ratio', 'purpose_', 'home_', 'emp_len_',  
              'borrower_score']  
outcome = 'outcome'  
X = pd.get_dummies(loan_data[predictors], prefix='', prefix_sep='',  
                  drop_first=True)  
y = loan_data[outcome]  
  
logit_reg = LogisticRegression(penalty='l2', C=1e42, solver='liblinear')  
logit_reg.fit(X, y)
```

In contrast to *R*, `scikit-learn` derives the classes from the unique values in *y* (*paid off* and *default*). Internally, the classes are ordered alphabetically. As this is the reverse order from the factors used in *R*, you will see that the coefficients are reversed. The `predict` method returns the class label and `predict_proba` returns the probabilities in the order available from the attribute `logit_reg.classes_`.



Generalized Linear Models

Generalized linear models (GLMs) are characterized by two main components:

- A probability distribution or family (binomial in the case of logistic regression)
- A link function—i.e., a transformation function that maps the response to the predictors (logit in the case of logistic regression)

Logistic regression is by far the most common form of GLM. A data scientist will encounter other types of GLMs. Sometimes a log link function is used instead of the logit; in practice, use of a log link is unlikely to lead to very different results for most applications. The Poisson distribution is commonly used to model count data (e.g., the number of times a user visits a web page in a certain amount of time). Other families include negative binomial and gamma, often used to model elapsed time (e.g., time to failure). In contrast to logistic regression, application of GLMs with these models is more nuanced and involves greater care. These are best avoided unless you are familiar with and understand the utility and pitfalls of these methods.



Predicted Values from Logistic Regression

The predicted value from logistic regression is in terms of the log odds: $\hat{Y} = \log(\text{Odds}(Y = 1))$. The predicted probability is given by the logistic response function:

$$\hat{P} = \frac{1}{1 + e^{-\hat{Y}}}$$

For example, look at the predictions from the model `logistic_model` in R:

```
pred <- predict(logistic_model)
summary(pred)
      Min.    1st Qu.    Median      Mean    3rd Qu.      Max.
-2.704774 -0.518825 -0.008539  0.002564  0.505061  3.509606
```

In *Python*, we can convert the probabilities into a data frame and use the `describe` method to get these characteristics of the distribution:

```
pred = pd.DataFrame(logit_reg.predict_log_proba(X),
                    columns=loan_data[outcome].cat.categories)
pred.describe()
```

Converting these values to probabilities is a simple transform:

```
prob <- 1/(1 + exp(-pred))
> summary(prob)
      Min.    1st Qu.    Median      Mean    3rd Qu.      Max.
0.06269  0.37313  0.49787  0.50000  0.62365  0.97096
```

The probabilities are directly available using the `predict_proba` methods in *scikit-learn*:

```
pred = pd.DataFrame(logit_reg.predict_proba(X),
                    columns=loan_data[outcome].cat.categories)
pred.describe()
```

These are on a scale from 0 to 1 and don't yet declare whether the predicted value is default or paid off. We could declare any value greater than 0.5 as default. In practice, a lower cutoff is often appropriate if the goal is to identify members of a rare class (see [“The Rare Class Problem” on page 223](#)).



Interpreting the Coefficients and Odds Ratios

One advantage of logistic regression is that it produces a model that can be scored to new data rapidly, without recomputation. Another is the relative ease of interpretation of the model, as compared with other classification methods. The key conceptual idea is understanding an *odds ratio*. The odds ratio is easiest to understand for a binary factor variable X :

$$\text{odds ratio} = \frac{\text{Odds}(Y = 1 | X = 1)}{\text{Odds}(Y = 1 | X = 0)}$$

This is interpreted as the odds that $Y = 1$ when $X = 1$ versus the odds that $Y = 1$ when $X = 0$. If the odds ratio is 2, then the odds that $Y = 1$ are two times higher when $X = 1$ versus when $X = 0$.

Why bother with an odds ratio rather than probabilities? We work with odds because the coefficient β_j in the logistic regression is the log of the odds ratio for X_j .

An example will make this more explicit. For the model fit in “**Logistic Regression and the GLM**” on page 210, the regression coefficient for `purpose_small_business` is 1.21526. This means that a loan to a small business compared to a loan to pay off credit card debt reduces the odds of defaulting versus being paid off by $\exp(1.21526) \approx 3.4$. Clearly, loans for the purpose of creating or expanding a small business are considerably riskier than other types of loans.

Figure 5-3 shows the relationship between the odds ratio and the log-odds ratio for odds ratios greater than 1. Because the coefficients are on the log scale, an increase of 1 in the coefficient results in an increase of $\exp(1) \approx 2.72$ in the odds ratio.

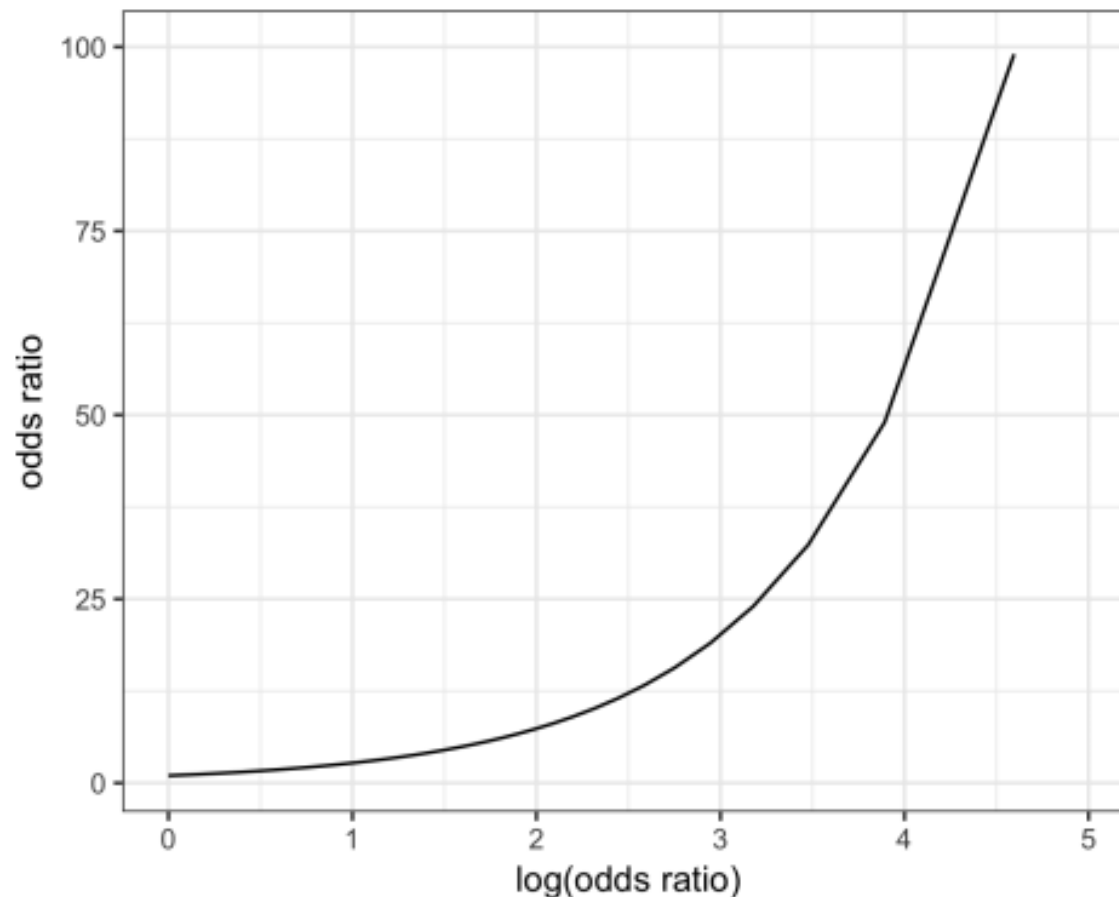


Figure 5-3. The relationship between the odds ratio and the log-odds ratio



로지스틱 회귀 > *Interpreting the Coefficients and Odds Ratios*

Logistic Regression

Odds ratios for numeric variables X can be interpreted similarly: they measure the change in the odds ratio for a unit change in X . For example, the effect of increasing the payment-to-income ratio from, say, 5 to 6 increases the odds of the loan defaulting by a factor of $\exp(0.08244) \approx 1.09$. The variable `borrower_score` is a score on the borrowers' creditworthiness and ranges from 0 (low) to 1 (high). The odds of the best borrowers relative to the worst borrowers defaulting on their loans is smaller by a factor of $\exp(-4.61264) \approx 0.01$. In other words, the default risk from the borrowers with the poorest creditworthiness is 100 times greater than that of the best borrowers!



Linear and Logistic Regression: Similarities and Differences

Linear regression and logistic regression share many commonalities. Both assume a parametric linear form relating the predictors with the response. Exploring and finding the best model are done in very similar ways. Extensions to the linear model, like the use of a spline transform of a predictor (see “[Splines](#)” on page 189), are equally applicable in the logistic regression setting. Logistic regression differs in two fundamental ways:

- The way the model is fit (least squares is not applicable)
- The nature and analysis of the residuals from the model

Fitting the model

Linear regression is fit using least squares, and the quality of the fit is evaluated using RMSE and R-squared statistics. In logistic regression (unlike in linear regression), there is no closed-form solution, and the model must be fit using *maximum likelihood estimation* (MLE). Maximum likelihood estimation is a process that tries to find the model that is most likely to have produced the data we see. In the logistic regression equation, the response is not 0 or 1 but rather an estimate of the log odds that the response is 1. The MLE finds the solution such that the estimated log odds best describes the observed outcome. The mechanics of the algorithm involve a quasi-Newton optimization that iterates between a scoring step (*Fisher's scoring*), based on the current parameters, and an update to the parameters to improve the fit.



Maximum Likelihood Estimation

Here is a bit more detail, if you like statistical symbols: start with a set of data $(X_1, X_2, \dots, X_n)$ and a probability model $P_\theta(X_1, X_2, \dots, X_n)$ that depends on a set of parameters θ . The goal of MLE is to find the set of parameters $\hat{\theta}$ that maximizes the value of $P_\theta(X_1, X_2, \dots, X_n)$; that is, it maximizes the probability of observing $(X_1, X_2, \dots, X_n)$ given the model P . In the fitting process, the model is evaluated using a metric called *deviance*:

$$\text{deviance} = -2 \log \left(P_{\hat{\theta}}(X_1, X_2, \dots, X_n) \right)$$

Lower deviance corresponds to a better fit.

Fortunately, most practitioners don't need to concern themselves with the details of the fitting algorithm since this is handled by the software. Most data scientists will not need to worry about the fitting method, other than understanding that it is a way to find a good model under certain assumptions.



Handling Factor Variables

In logistic regression, factor variables should be coded as in linear regression; see [“Factor Variables in Regression” on page 163](#). In *R* and other software, this is normally handled automatically, and generally reference encoding is used. All of the other classification methods covered in this chapter typically use the one hot encoder representation (see [“One Hot Encoder” on page 242](#)). In *Python*'s *scikit-learn*, it is easiest to use one hot encoding, which means that only $n - 1$ of the resulting dummies can be used in the regression.



로지스틱 회귀 > Assessing the Model

Logistic Regression

Assessing the Model

Like other classification methods, logistic regression is assessed by how accurately the model classifies new data (see “Evaluating Classification Models” on page 219). As with linear regression, some additional standard statistical tools are available to examine and improve the model. Along with the estimated coefficients, R reports the standard error of the coefficients (SE), a z-value, and a p-value:

```
summary(logistic_model)
```

Call:

```
glm(formula = outcome ~ payment_inc_ratio + purpose_ + home_ +  
    emp_len_ + borrower_score, family = "binomial", data = loan_data)
```

Deviance Residuals:

Min	1Q	Median	3Q	Max
-2.51951	-1.06908	-0.05853	1.07421	2.15528

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.638092	0.073708	22.224	< 2e-16 ***
payment_inc_ratio	0.079737	0.002487	32.058	< 2e-16 ***
purpose_debt_consolidation	0.249373	0.027615	9.030	< 2e-16 ***
purpose_home_improvement	0.407743	0.046615	8.747	< 2e-16 ***
purpose_major_purchase	0.229628	0.053683	4.277	1.89e-05 ***
purpose_medical	0.510479	0.086780	5.882	4.04e-09 ***
purpose_other	0.620663	0.039436	15.738	< 2e-16 ***
purpose_small_business	1.215261	0.063320	19.192	< 2e-16 ***
home_OWN	0.048330	0.038036	1.271	0.204
home_RENT	0.157320	0.021203	7.420	1.17e-13 ***
emp_len_ > 1 Year	-0.356731	0.052622	-6.779	1.21e-11 ***
borrower_score	-4.612638	0.083558	-55.203	< 2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 62857 on 45341 degrees of freedom
 Residual deviance: 57515 on 45330 degrees of freedom
 AIC: 57539

Number of Fisher Scoring iterations: 4



로지스틱 회귀 > Assessing the Model

Logistic Regression

The package `statsmodels` has an implementation for generalized linear model (GLM) that provides similarly detailed information:

```
y_numbers = [1 if yi == 'default' else 0 for yi in y]
logit_reg_sm = sm.GLM(y_numbers, X.assign(const=1),
                      family=sm.families.Binomial())
logit_result = logit_reg_sm.fit()
logit_result.summary()
```

Interpretation of the p-value comes with the same caveat as in regression and should be viewed more as a relative indicator of variable importance (see [“Assessing the Model” on page 153](#)) than as a formal measure of statistical significance. A logistic regression model, which has a binary response, does not have an associated RMSE or R-squared. Instead, a logistic regression model is typically evaluated using more general metrics for classification; see [“Evaluating Classification Models” on page 219](#).

Many other concepts for linear regression carry over to the logistic regression setting (and other GLMs). For example, you can use stepwise regression, fit interaction terms, or include spline terms. The same concerns regarding confounding and correlated variables apply to logistic regression (see [“Interpreting the Regression Equation” on page 169](#)). You can fit generalized additive models (see [“Generalized Additive Models” on page 192](#)) using the `mgcv` package in R:

```
logistic_gam <- gam(outcome ~ s(payment_inc_ratio) + purpose_ +
                    home_ + emp_len_ + s(borrower_score),
                    data=loan_data, family='binomial')
```

The formula interface of `statsmodels` also supports these extensions in *Python*:

```
import statsmodels.formula.api as smf
formula = ('outcome ~ bs(payment_inc_ratio, df=4) + purpose_ + ' +
          'home_ + emp_len_ + bs(borrower_score, df=4)')
model = smf.glm(formula=formula, data=loan_data, family=sm.families.Binomial())
results = model.fit()
```



로지스틱 회귀 > Assessing the Model

Logistic Regression

Analysis of residuals

One area where logistic regression differs from linear regression is in the analysis of the residuals. As in linear regression (see [Figure 4-9](#)), it is straightforward to compute partial residuals in R:

```
terms <- predict(logistic_gam, type='terms')
partial_resid <- resid(logistic_model) + terms
df <- data.frame(payment_inc_ratio = loan_data[, 'payment_inc_ratio'],
                 terms = terms[, 's(payment_inc_ratio)'],
                 partial_resid = partial_resid[, 's(payment_inc_ratio)'])
ggplot(df, aes(x=payment_inc_ratio, y=partial_resid, solid = FALSE)) +
  geom_point(shape=46, alpha=0.4) +
  geom_line(aes(x=payment_inc_ratio, y=terms),
            color='red', alpha=0.5, size=1.5) +
  labs(y='Partial Residual')
```

The resulting plot is displayed in [Figure 5-4](#). The estimated fit, shown by the line, goes between two sets of point clouds. The top cloud corresponds to a response of 1 (defaulted loans), and the bottom cloud corresponds to a response of 0 (loans paid off). This is very typical of residuals from a logistic regression since the output is binary. The prediction is measured as the logit (log of the odds ratio), which will always be some finite value. The actual value, an absolute 0 or 1, corresponds to an infinite logit, either positive or negative, so the residuals (which get added to the fitted value) will never equal 0. Hence the plotted points lie in clouds either above or below the fitted line in the partial residual plot. Partial residuals in logistic regression, while less valuable than in regression, are still useful to confirm nonlinear behavior and identify highly influential records.

There is currently no implementation of partial residuals in any of the major *Python* packages. We provide *Python* code to create the partial residual plot in the accompanying source code repository.

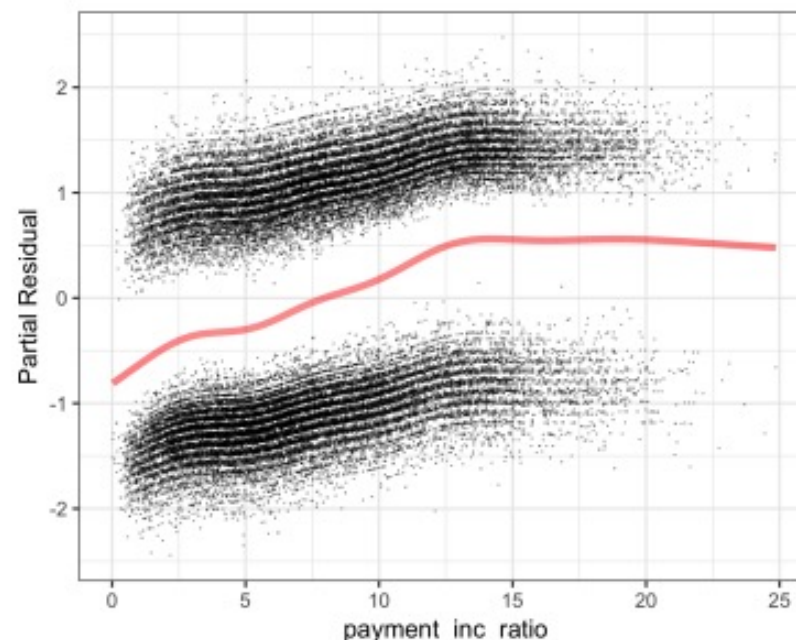


Figure 5-4. Partial residuals from logistic regression



로지스틱 회귀 > *Assessing the Model*

Logistic Regression



Some of the output from the summary function can effectively be ignored. The dispersion parameter does not apply to logistic regression and is there for other types of GLMs. The residual deviance and the number of scoring iterations are related to the maximum likelihood fitting method; see “**Maximum Likelihood Estimation**” on page 215.

Key Ideas

- Logistic regression is like linear regression, except that the outcome is a binary variable.
- Several transformations are needed to get the model into a form that can be fit as a linear model, with the log of the odds ratio as the response variable.
- After the linear model is fit (by an iterative process), the log odds is mapped back to a probability.
- Logistic regression is popular because it is computationally fast and produces a model that can be scored to new data with only a few arithmetic operations.

Q&A

질의응답 (5분)





다음 주 배울 내용

Contents for Next Week

분류

- 분류모형 평가
- 불균형 데이터 처리
- Etc

통계적 머신러닝

끝. 감사합니다.

수업 듣느라 수고하셨습니다.

